
Chapter 16 Linked Lists

16.1 Working with Nodes

- Constructing a List 
- List Basics 
- Manipulating Nodes 
- Traversing a List 

16.2 A Linked List Class

- Simple `LinkedList` 
- Appending `add` 
- The Middle of the List 

16.3 A Complex List Operation

- Inchworm Approach 

16.4 An `IntList` Interface

16.5 `LinkedList<E>`

- Linked List Variations 

- [Linked List Iterators](#) 

- [Other Code Details](#) 

Introduction

In the previous chapter, we explored how to build a list structure using an array as the underlying storage mechanism. In this chapter we will explore a different structure known as a linked list. Linked lists serve as a useful contrast to arrays because their strengths and weaknesses are completely opposite. And just as Java has an `ArrayList<E>` class as part of the collections framework, it also has a `LinkedList<E>` class.

Even though Java provides an off-the-shelf implementation of a linked list, it is useful to study how it is implemented because it will give you good insights into the properties of linked lists. It is also important for you to understand the concept of a linked structure. Almost all data structures are implemented using some combination of arrays or linking or both, which means that it is important to understand each approach.

We will once again develop a structure for storing simple `int` values, as we did with the development of `ArrayIntList` in the last chapter. Our new structure will be called `LinkedList`. Keeping the data simple will allow us to focus on the data structure issues and to learn how to manipulate linked lists in general. Then we will explore some issues related to interfaces. Finally, at the end of the chapter we will explore how to turn the fairly simple `LinkedList` into a more general `LinkedList<E>` class.

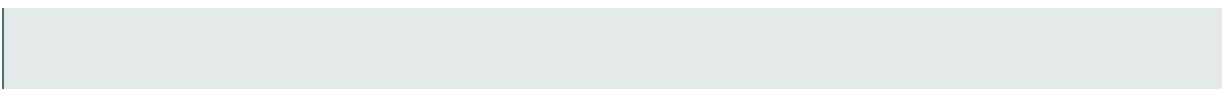
16.1 Working with Nodes

Because arrays are stored in one large contiguous block of memory, we can quickly locate any particular element of the array. Recall that this ability to quickly jump around in the array is called random access. The weaknesses of the array approach are that we can't easily insert values in or remove values from the middle without shifting other values, and we can't easily enlarge the size of the structure without constructing a brand-new array with a larger capacity.

The linked list structure has the opposite properties. It is not a random access structure, so it is not easy to jump around in the structure. It has the kind of sequential access we associate with a cassette tape. If you are playing a tape and want to skip forward 10 songs, you have to fast forward through the songs. You can't quickly skip to the position where you want to be on the tape. Linked lists have this property as well.

But where arrays are weak, linked lists are strong. We can quickly insert values in or delete values from the middle of a linked list without any shifting. It is also easy to make the list larger or smaller.

Linked lists are composed of individual elements called *nodes*.

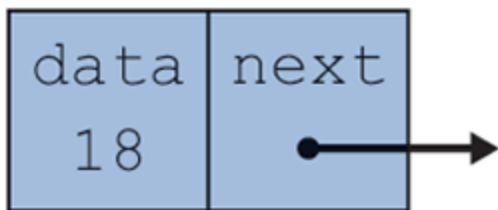


Node

A single element of a structure such as a linked list; each node contains one data value.

A node is like a Lego building block. It looks unimpressive by itself, but once you put a bunch of them together, it can form an interesting structure.

A basic list node looks like this:



It's an object with two fields: one for storing a single item of data and one for storing a reference to the next node in the list. To store a list of integer values we'd declare the node class as follows:

```
public class ListNode {  
    public int data;  
    public ListNode next;  
}
```

This class does not produce a nicely encapsulated object with private fields, but the technique of using public fields is the usual approach to defining nodes. In the next section we'll discuss why it is acceptable to avoid encapsulation in this case.

This is a recursive data structure. The `ListNode` class is defined in terms of itself because it has a field of type `ListNode`. As a result, it is often possible to solve linked list programming problems more effectively by writing recursive methods, as described in [Chapter 12](#) .

Constructing a List

Let's begin by constructing a series of nodes that store the sequence of values `[3, 7, 12]`. There are three values, which means you'll need three nodes that are linked together. When you're creating linked lists, if you keep a reference to the front of the list, then you can get to anything in the list. You'll usually use a single variable of type `ListNode` that refers to (or points to) the front of the list. You begin with this declaration:

```
ListNode list;
```

The variable `list` is not itself a node. It's a variable that is capable of referring to a node. You haven't yet given it a value, so you would

draw the following picture for memory at this point in the program:



The “?” will be replaced with a reference to a node, which means that this box does not have a `data` field or a `next` field. It’s a box where you can store a reference to such an object. You don’t have an actual node yet. To understand this distinction between an object and a variable that stores a reference to an object, you might want to review the section on reference semantics in [Chapter 7](#).

The way to get an actual node is to call `new`:

```
list = new ListNode();
```

This call constructs a new node and tells Java to have the variable `list` refer to it:



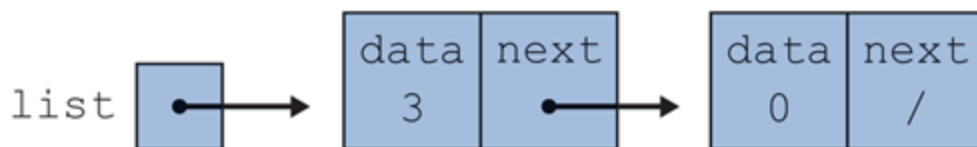
Recall that when objects are constructed, Java initializes the fields to the zero-equivalent for the type. That is why in the node above the

`data` field has the value `0` and the `next` field has the value `null`.
Notice that we use a slash through the box to indicate a `null` value.

What do you want to do with the node you have constructed? You want to store `3` in its `data` field (`list.data`) and you want its `next` field to point to a new node:

```
list.data = 3;  
list.next = new ListNode();
```

The addition of these lines of code makes our linked list appear as follows:

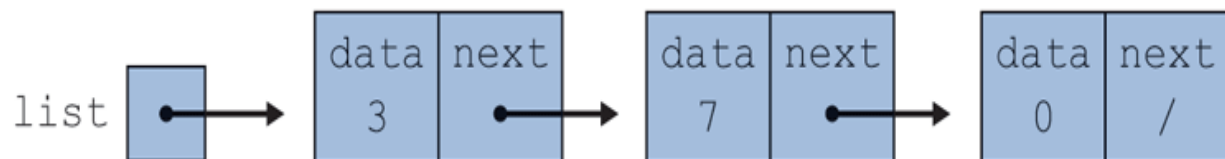


When you program linked lists, you have to understand how to access the different elements of the structure. The variable `list` stores a reference to the first node. You can get inside that node with the dot notation (`list.data` and `list.next`). So `list.next` is the way to refer to the `next` box of the first node. You wrote code to assign it to refer to a new node, which is why `list.next` is pointing at this second node.

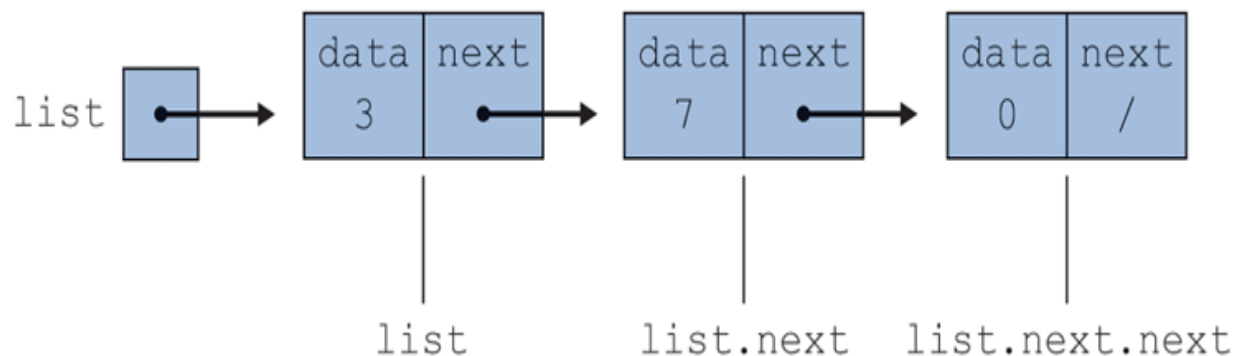
Now you want to assign the second node's data field (`list.next.data`) to the value `7` and assign the second node's `next` field to refer to a third node:

```
list.next.data = 7;  
list.next.next = new ListNode();
```

Now the linked list looks like this:



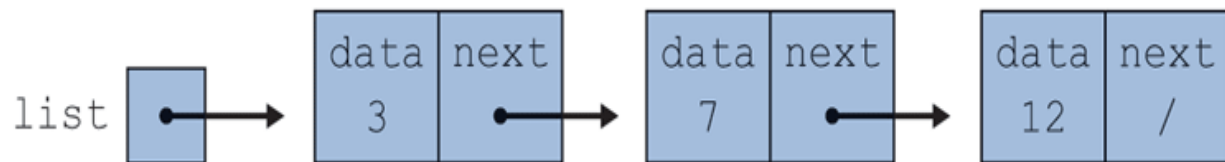
Again, pay close attention to the difference between `list`, `list.next`, and `list.next.next`, and remember which box is associated with each of these:



Finally, you want to set the `data` field of this third node to `12` (stored in `list.next.next.data`) and you want to set its `next` field to `null`:

```
list.next.next.data = 12;  
list.next.next.next = null;
```

Our linked list now looks like this:



The assignment of the `next` field to `null` is actually unnecessary because Java initializes it to that value, but it's not a bad idea to be explicit about the value that you want to store in the field.

The dot notation is often very confusing for novices, so it is worth pausing for a moment to make sure that you understand the different combinations and the elements to which they refer. [Table 16.1](#) includes each of the possibilities for the preceding diagram.

Table 16.1 Referring to Elements of a Sample List

Here is a complete program that includes all this code for constructing a three-element list along with code that prints the list:

```

1 // Constructs and prints the list [3, 7, 12] by setting
each
2 // field of each node.
3
4 public class Construct1 {
5     public static void main(String[] args) {
6         ListNode list = new ListNode();
7         list.data = 3;
8         list.next = new ListNode();
9         list.next.data = 7;
10        list.next.next = new ListNode();
11        list.next.next.data = 12;
12        list.next.next.next = null;
13        System.out.println(list.data + " " + list.next.data
+ " "
14                               + list.next.next.data);
15    }
16 }

```

The program produces the following output:

```
3 7 12
```

Obviously, this program represents a very tedious way to manipulate a list. It's much better to write code that involves loops to manipulate lists. But it takes a while to get used to this idea, so we're first going to practice doing some raw list operations without a loop.

List Basics

Our previous version of the node class has just two fields:

```
public class ListNode {  
    public int data;  
    public ListNode next;  
}
```

In general, we like to keep the node class simple, so we don't want to add much to this. But it is a good idea to include some constructors:

```
1 // ListNode is a class for storing a single node of a  
linked  
2 // list. This node class is for a list of integer values.  
3  
4 public class ListNode {
```

```

5     public int data;        // data stored in this node
6     public ListNode next;   // link to next node in the list
7
8     // post: constructs a node with data 0 and null link
9     public ListNode() {
10         this(0, null);
11     }
12
13     // post: constructs a node with given data and null
link
14     public ListNode(int data) {
15         this(data, null);
16     }
17
18     // post: constructs a node with given data and given
link
19     public ListNode(int data, ListNode next) {
20         this.data = data;
21         this.next = next;
22     }
23 }

```

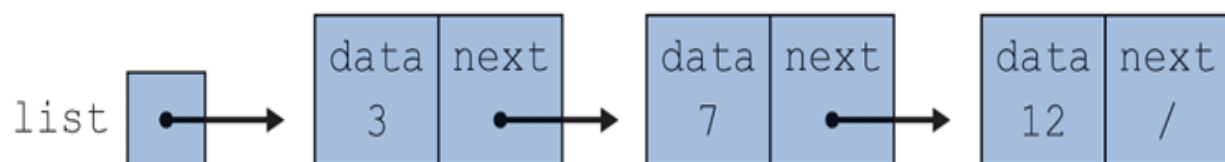
Like the other classes we've seen, this class has one “real” constructor (the one that takes two arguments). The other two constructors use the `this(...)` notation to call the third constructor with default values (0 for the data, `null` for next). In the new version of the class, it is possible to write a single statement to construct the three-element list we have been studying:

```

1 // Constructs and prints the list [3, 7, 12]. This version
uses
2 // node constructors rather than setting fields of each
node.
3
4 public class Construct2 {
5     public static void main(String[] args) {
6         ListNode list = new ListNode(3,
7             new ListNode(7, new ListNode(12)));
8         System.out.println(list.data + " " + list.next.data
+ " "
9             + list.next.next.data);
10    }
11 }

```

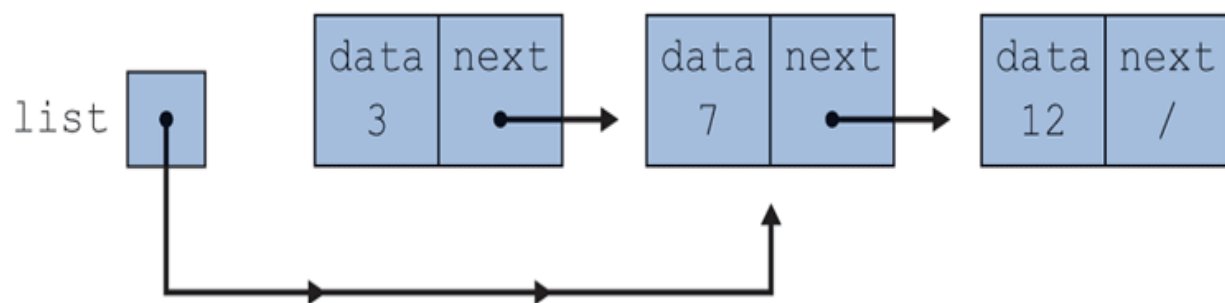
In some programming languages you have to explicitly free up memory when you are no longer using it. That is not the case in Java. In Java you can simply stop referring to a node when you no longer want to use it. For example, consider the following three-element list that we have constructed:



How would you get rid of the first node? You can reassign the variable `list` to point to the node that comes after it. In effect, you leapfrog over the node that stores `3` and instead have `list` point to the node that stores `7` by writing the following line of code:

```
list = list.next;
```

Now the linked list contains the following:



Now the variable `list` is pointing at a two-element list because it points at the node storing `7`, which in turn points to the node that stores `12`. The node that stores `3` is no longer pointed to by any variable.

What happens to a node when no variable points to it? As we mentioned at the end of the previous chapter, the Java runtime system periodically invokes the garbage collector to look for objects like this node and to reclaim the space so that it can be used again.

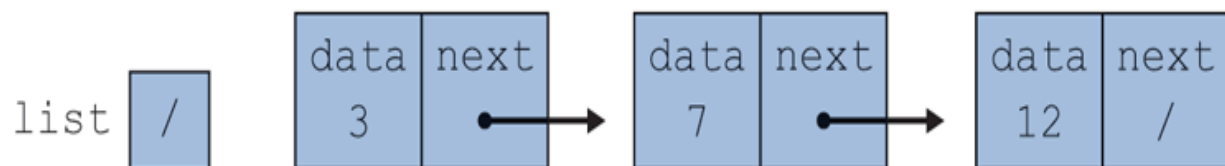
Java programmers enjoy the convenience of relying on this process, which is sometimes called *automatic garbage collection*.

A good analogy is to think of each node as a helium balloon and the arrows as the strings that we use to hold on to the balloons. If you let go of a string, then the balloon floats away. But the garbage collector will find all of those stray balloons and reclaim the space.

As one final example, consider what happens if you were to reset the `list` variable to `null`:

```
list = null;
```

Our linked list would look like this:



The value `null` is used to represent the empty list. There are three nodes floating around from the work we did earlier, but those are like helium balloons that have floated away because their strings were let go. They will eventually be recycled by the garbage collector.

You have to be careful about manipulating an empty list. For example, if the variable `list` is `null`, and you execute code that refers to `list.data`, Java will halt your program by throwing a `NullPointerException`. You will quickly discover that `NullPointerException` is one of the most common problems that you run into as you try to debug your linked list code. Just keep in mind that Java throws that exception when you attempt to dereference a `null` value. In other words, it occurs when you ask for `ptr.fieldName` where `ptr` has the value `null`. When Java throws the exception, it will show you the exact line number where it occurred. Look at that line of code carefully to find all occurrences of the dot notation, because one of them involves a `null` value.

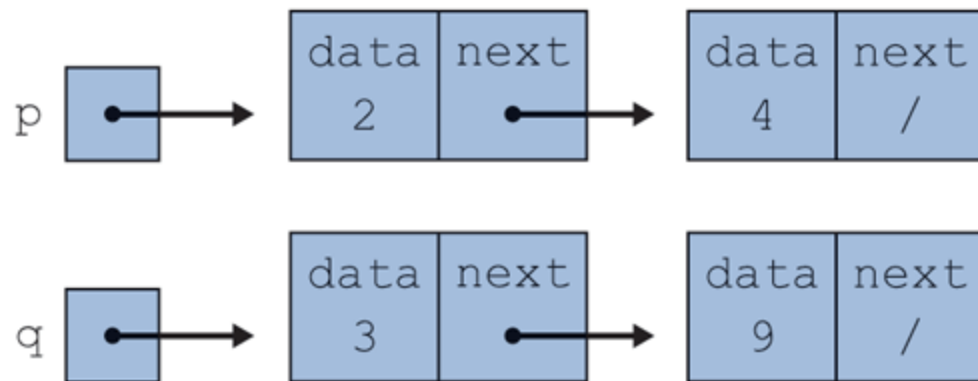
Manipulating Nodes



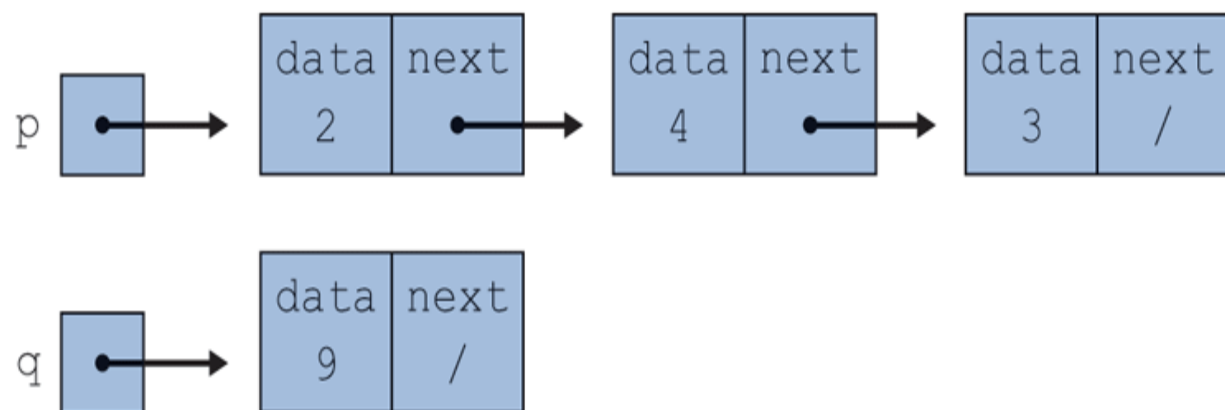
In the next section, we will explore how to use loops to write more generalized code, but first it is useful to practice basic node manipulation. A good way to practice is to make up an exercise that

involves a “before” picture and an “after” picture. The challenge is to write code that gets you from the first state to the second state.

As an example, suppose that you have two variables of type `ListNode` called `p` and `q` and that the following is the “before” situation:

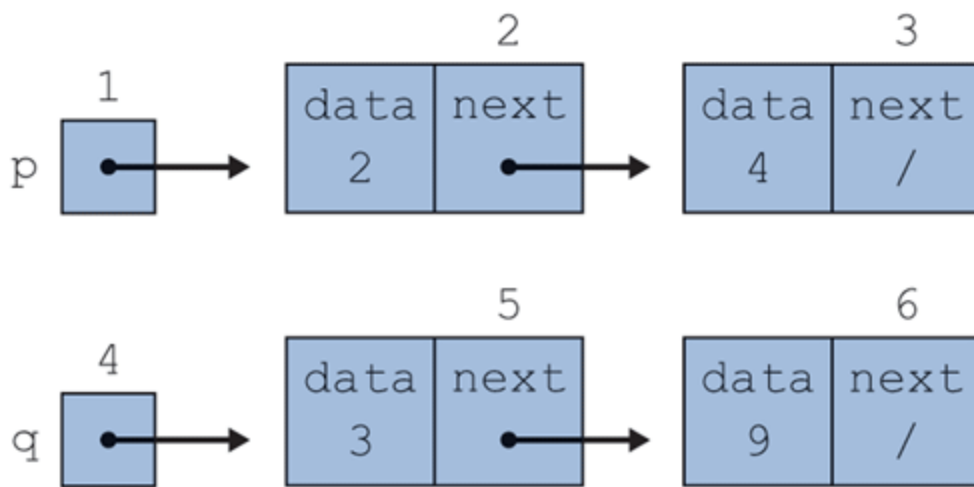


Suppose that you want to get to the following “after” situation:



In order to accomplish this, you will have to rearrange the links of these two lists. As a starting point, think about how many variables of

type `ListNode` there are. You might say two because there are two named variables, `p` and `q`. Or you might say four because you notice that each of the four nodes has a field of type `ListNode`. Actually, there are six different variables of type `ListNode`. The following diagram numbers each of the six variables:

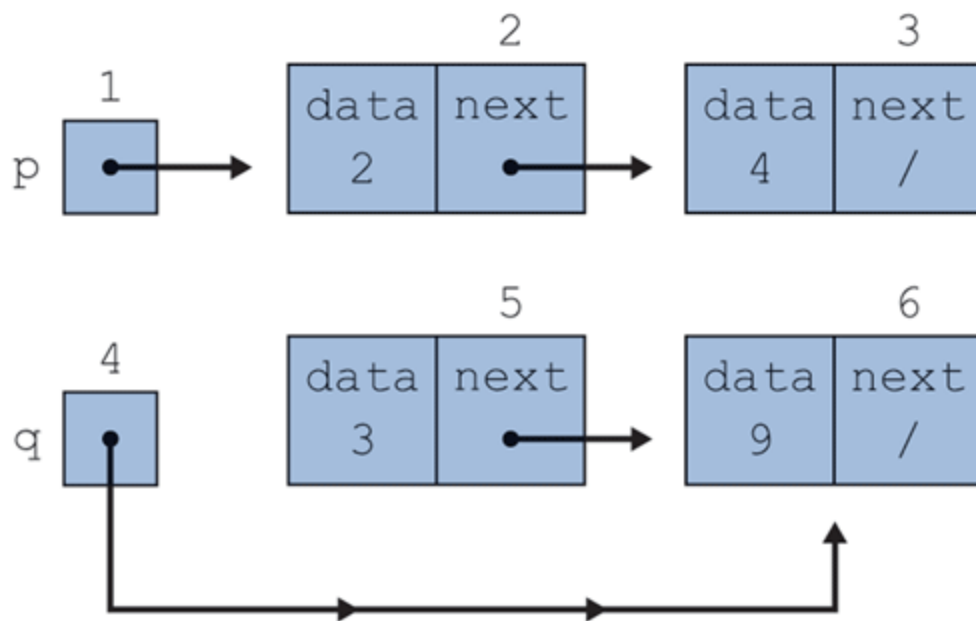


Having the variables numbered makes it easier to discuss the task at hand. Which of these variables has to change in value to get from the before picture to the after picture? The boxes numbered 3, 4, and 5 have to change. If we change them appropriately, we'll be done.

We have to be careful about how we change the links. The order can be important. For example, suppose we start by changing box 4 (the variable `q`). In the final situation, it's supposed to point at the node with 9 in it. We can accomplish this by leapfrogging over the current node to which it is pointing:

```
q = q.next;
```

But if we start with this change, what happens to the node that contains 3? We lose track of it, as the following picture indicates:



Once we have lost track of the node containing 3, we have no way to get back to it. It's like a helium balloon that has floated away because we let go of the string.

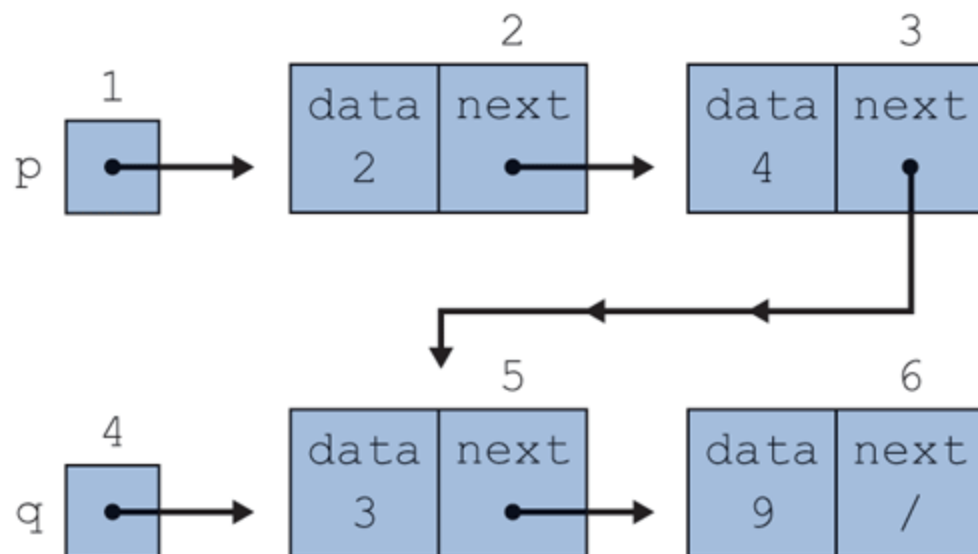
This is a common problem in linked list programming that you have to consider carefully. One solution is to introduce a temporary variable that would keep track of the node to which the variable `q` used to point (the node containing 3). Often, though, we can instead

solve the problem by carefully choosing the order of the changes that we make.

Of the three values we have to change to solve this problem, the one that is safe to change is box 3 because it's currently `null`. We begin by setting it to point to the node containing 3:

```
p.next.next = q;
```

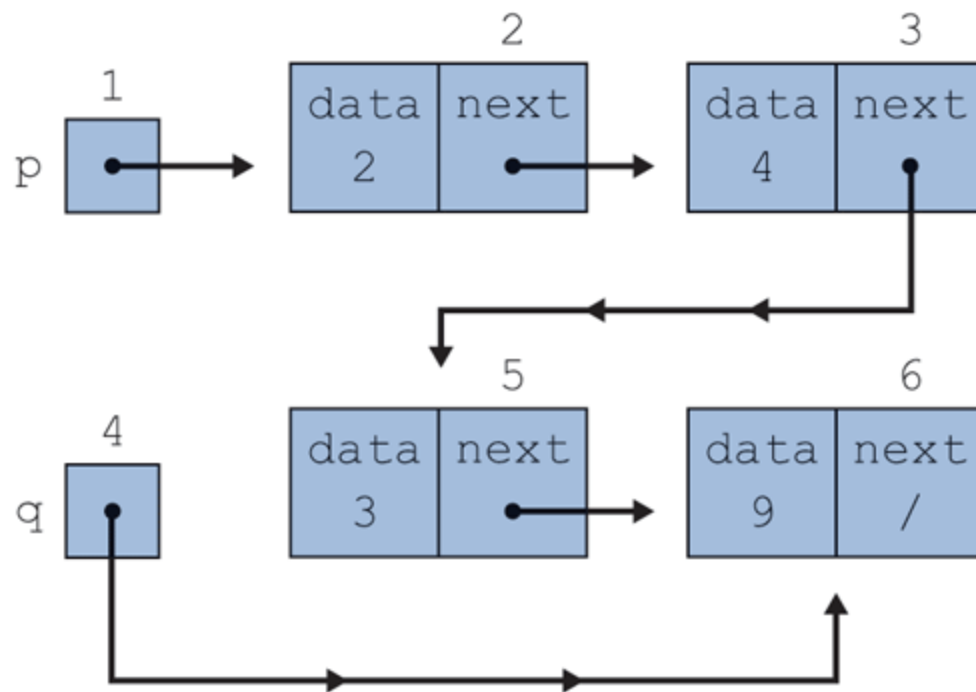
Our linked list now looks like this:



Now that we've used the value of box 4 to reset box 3, we can reset box 4. It's supposed to point to the node that has 9 in it. Now we can leapfrog over the current node to which it is pointing:

```
q = q.next;
```

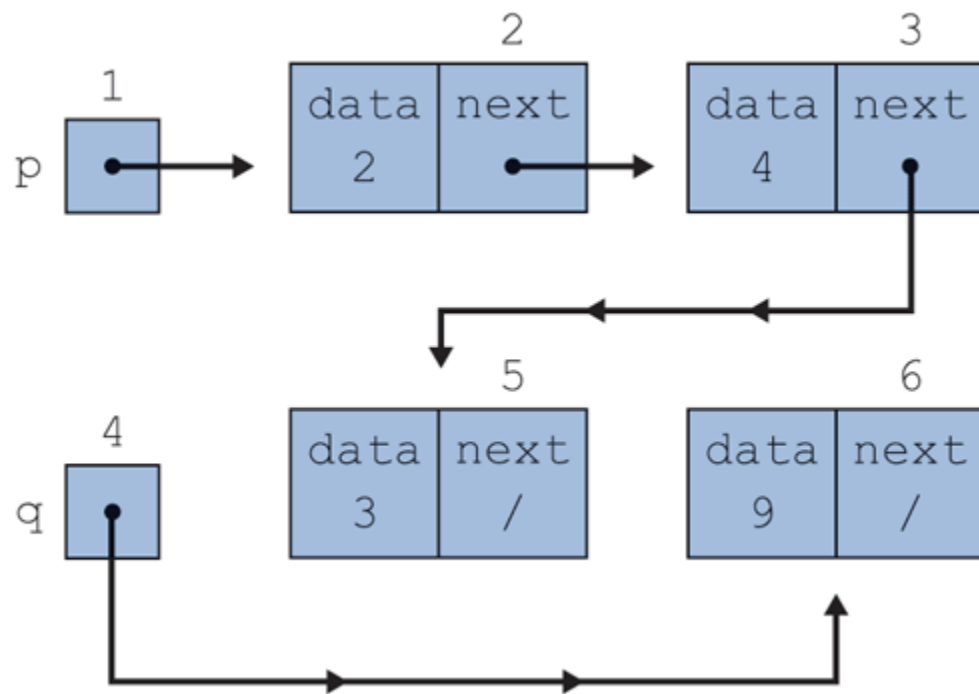
Our linked list now looks like this:



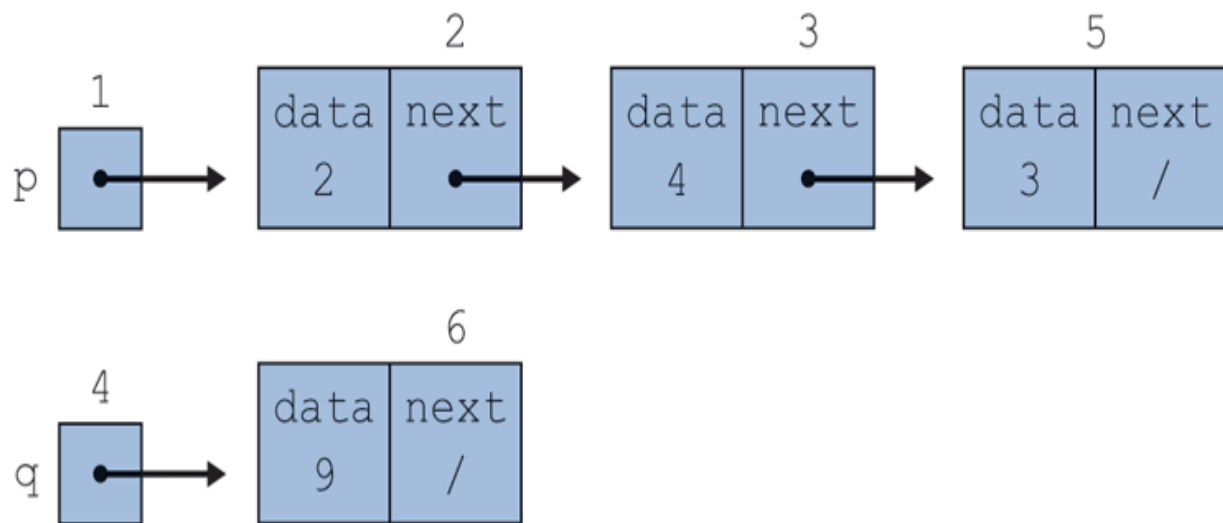
Now we just have to reset box 5. In the original picture, we would have referred to box 5 as `q.next`, but now that we've changed the value of `q`, we have to change the way we refer to this box. We can still get to box 5 by starting with the variable `p`:

```
p.next.next.next = null;
```

Our linked list now looks like this:



We can simplify this diagram by redrawing the three nodes that are now part of the first list all in a row. Keep in mind, though, that the nodes themselves haven't moved around in memory. We have only rearranged various links. But we can more easily understand the structure if we draw the picture in this way:



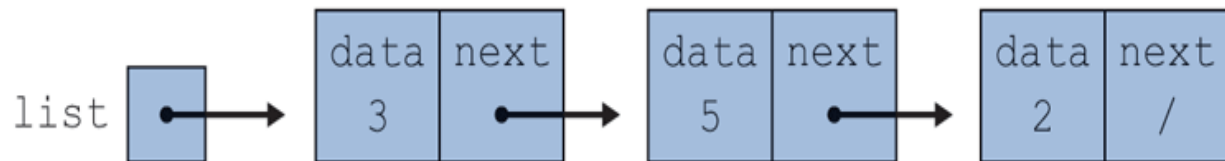
And now we can see that this diagram matches the “after” picture we were given. The three lines of code that are needed to get from the initial state to the final state are as follows:

```
p.next.next = q;  
q = q.next;  
p.next.next.next = null;
```

Obviously, this process can be very confusing. It is essential to draw pictures to keep track of what is pointing where and what is going on when this code executes. It’s the only way to master linked list code.

Traversing a List

Consider the problem of printing each value in a list on a different line. For example, suppose we have a variable called `list` that stores a reference to the list `[3, 5, 2]`:

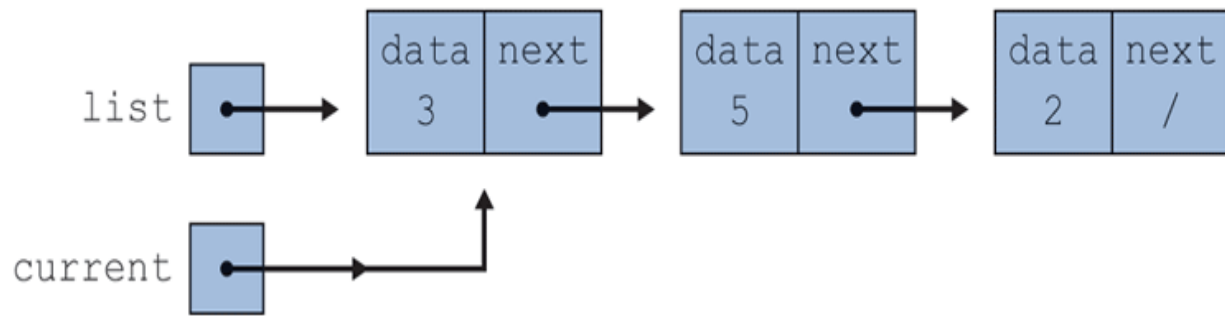


Using the techniques from the previous section you could refer to each of the three data fields: `list.data` (3), `list.next.data` (5), and `list.next.next.data` (2). This approach can work for very short lists, but obviously won't work when you have hundreds or thousands of nodes to process. In that case, you'd want to write a loop.

You have just one variable to work with, the variable `list`, so that's clearly where you have to start. You could use it to move along the list and print things out, but then you would lose the original value of the variable, which would mean that you would have lost the list. Instead, it's better to declare a local variable of type `ListNode` that you use to access the different nodes of the list:

```
ListNode current = list;
```

You can use any variable name you like, although it is common in linked list programming to use a name like `current` or `curr`. The preceding line of code initializes `current` to point to the same value as `list` (the first node in the list):



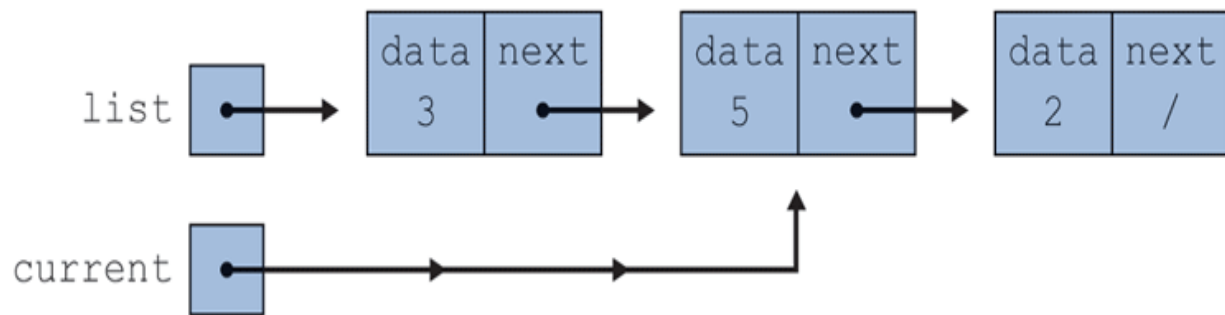
You want a loop that prints the various values, and you want it to keep going as long as there is more data to print. So how do you structure your loop? The variable `current` will refer to each different node in turn. The final node has the value `null` in its `next` field, so eventually the variable `current` will become equal to `null` and that's when you know your loop is finished. Thus, your basic loop structure will be as follows:

```
ListNode current = list;
while (current != null) {
    process next value.
    move current forward.
}
```

To process a node, you need to print out its value, which you can get from `current.data`, and you need to move `current` to the next node over. The position of the next node is stored in `current.next`, so moving to that next node involves resetting `current` to `current.next`:

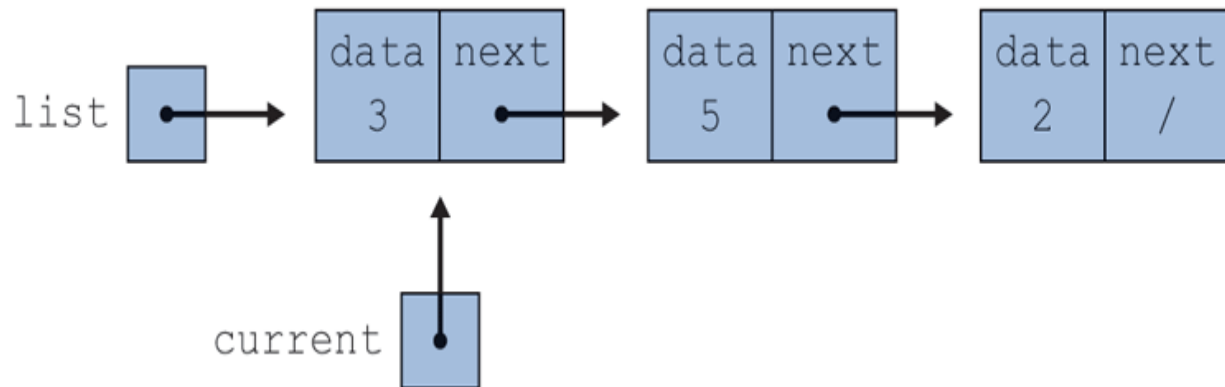
```
ListNode current = list;
while (current != null) {
    System.out.println(current.data);
    current = current.next;
}
```

The first time through this loop, `current` is referring to the node with the `3` in it. It prints this value and then resets `current`, which causes `current` to refer to (or point to) the second node in the list:

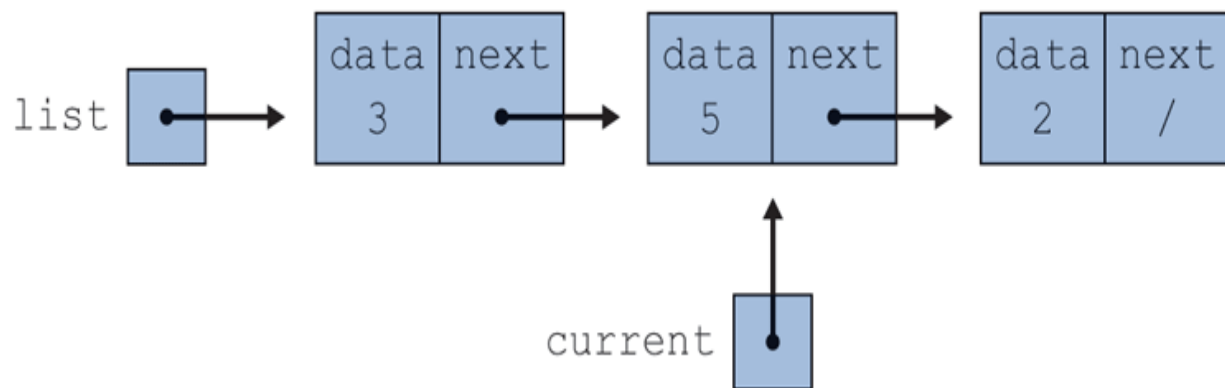


Some people prefer to visualize this differently. Instead of envisioning the variable `current` sitting still while its arrow moves,

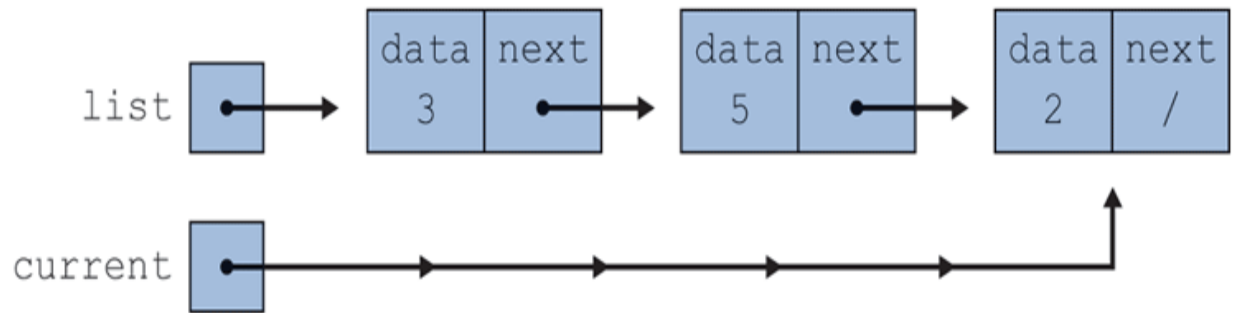
some people prefer to envision the variable itself moving. So, for the initial situation, you draw the following picture:



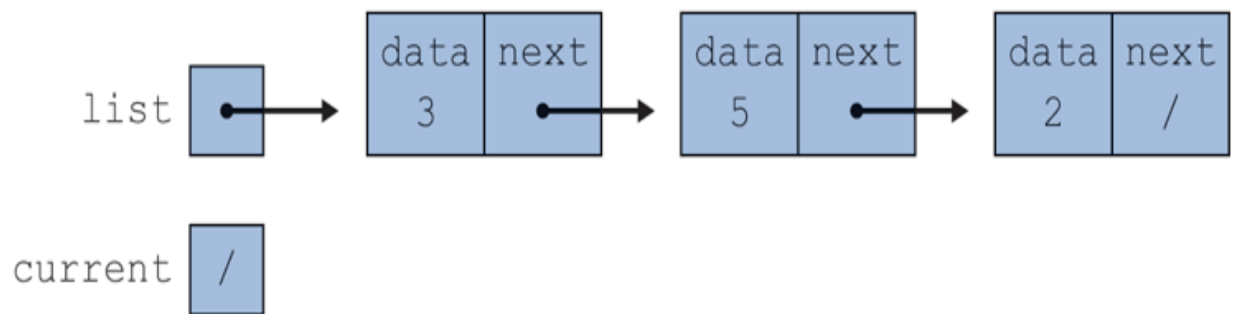
After execution of the statement `current = current.next`, the linked list would look like this:



Either way of thinking about this scenario works. Because in this new situation the variable `current` is not `null`, you once again enter the loop, print out `current.data` (which is now 5), and move `current` along again:



Once again `current` is not `null`, so you enter the loop a third time, print the value of `current.data` (2), and reset `current`. But this time `current.next` has the value `null`, so when you reset `current`, the computer's memory will look like this:



Because `current` has become `null`, when you break out of the loop the program has produced the following output:

```
3
5
2
```

The corresponding array code would look like this:

```
int i = 0;
while (i < size) {
    System.out.println(elementData[i]);
    i++;
}
```

Assuming that you have some comfort with array-style programming, this example might give you some useful insight into linked list programming. There are direct parallels in terms of typical code, as shown in [Table 16.2](#).

Table 16.2 Array/Linked List Equivalents

You may recall that in [Chapter 7](#) we introduced a standard array traversal approach:

```
for (int i = 0; i < <array>.length; i++) {
    <do something with array[i]>;
}
```

You can write a similar standard linked list traversal pattern:

```
<temp variable> = <front of list>;  
while (<temp variable> != null) {  
    <do something with <temp variable>.data>;  
    <temp> variable> = <temp variable>.next;  
}
```

Knowing that we like to use `for` loops for array processing, you can imagine writing `for` loops to process linked lists as well. Your previous code could be rewritten as the following code:

```
for (ListNode current = list; current != null; current =  
current.next) {  
    System.out.println(current.data);  
}
```

Some people like to write their list code this way, but most tend to use `while` loops. It's an issue of personal taste.

16.2 A Linked List Class



In this section, we will explore how to write a class called `LinkedList` that will be a parallel of the `ArrayList` class of [Chapter 15](#).

Simple `LinkedList`

So far we have been discussing the low-level details of how to manipulate the nodes of a class. In general, we want to provide a potential client with a simple interface that doesn't require the client to understand these low-level details. For example, the `ArrayList` class that we examined in [Chapter 15](#) uses an array as its underlying structure, but a client of the class never has to know that. In a similar way, we'd like to define a `LinkedList` class that a

client can access without having to understand that it is implemented by means of a linked list of nodes.

First we have to consider the set of fields that we need. As we've seen, we can get to any element in the list as long as we have a reference to the front of the list. So at a minimum we need a reference to the front of the list:

```
public class LinkedList {  
    private ListNode front;  
    ...  
}
```

There are several other fields we could add to improve efficiency. Two of the most commonly added fields keep track of the length of the list and keep a reference to the back of the list. But for this first version, we will keep it simple and store just a reference to the front of the list.

Notice that for this class, the field `front` is declared to be private. We do that to guarantee that the class is well encapsulated. But what about those public fields in the node class? In general public fields are a bad idea. But they're not of great concern in this case because we're going to make sure that only our `LinkedList` object will ever manipulate individual nodes. By the time we complete this program, we will have two classes: one for individual nodes of a list and one

for the list itself. We'll be careful to have a clean, well-encapsulated list object, but we don't have to worry about doing the same thing for the node class.

Some people prefer to encapsulate even node objects to keep things simple, but if you want to learn how complex classes are written, it is important to understand this convention. When we are writing code to interact with a client, we want everything to be completely encapsulated. But when we are writing code that is seen only by the implementation, we can be more relaxed. As an analogy, think of how you behave in public versus in your own home. In your own home you might walk around in a bathrobe and slippers because you know that only you and your family have access to your home. You wouldn't tend to wear a bathrobe and slippers out in public. As another example, think of the post office boxes you find at a post office. Those post office boxes are locked for security, just like an encapsulated object, because anyone has access to the public post office. But in a corporate office setting, mailboxes often are not locked because only a limited number of people have access to them and, therefore, people are willing to be more informal.

Later in the chapter, we will see an even better way of handling this case by including the node class as a static inner class. But for now we'll work with the two classes to keep things simple.

As a first exercise, let's add a method to our class that will allow us to examine the contents of the list. We saw that code like the following could be used to print a list:

```
ListNode current = list;
while (current != null) {
    System.out.println(current.data);
    current = current.next;
}
```

We could turn that into a method of the class by initializing the local variable to the value of our field `front`:

```
public void print() {
    ListNode current = front;
    while (current != null) {
        System.out.println(current.data);
        current = current.next;
    }
}
```

But as we saw in [Chapter 15](#), it is a better idea to make this a `toString` method that returns a `String` representation of the list. For the `ArrayIntList` class we wrote the following method:

```
public String toString() {
```

```

    if (size == 0) {
        return "[]";
    } else {
        String result = "[" + elementData[0];
        for (int i = 1; i < size; i++) {
            result += ", " + elementData[i];
        }
        result += "];";
        return result;
    }
}

```

The code is very similar for the `LinkedList` class because we will have the same special case for an empty list and the same fencepost loop to make sure that values are separated by commas. If we translate each array operation into its linked list equivalent, we end up with the following code:

```

public String toString() {
    if (front == null) {
        return "[]";
    } else {
        String result = "[" + front.data;
        ListNode current = front.next;
        while (current != null) {
            result += ", " + current.data;

```

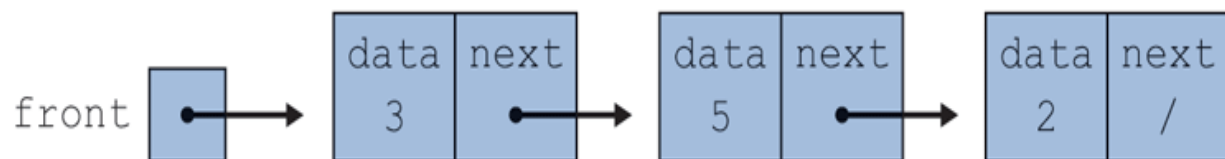
```
        current = current.next;
    }

    result += "];"
    return result;
}
}
```

Notice that in this code we initialize `current` to `front.next` because we handle the data for the first node outside the loop.

Appending `add`

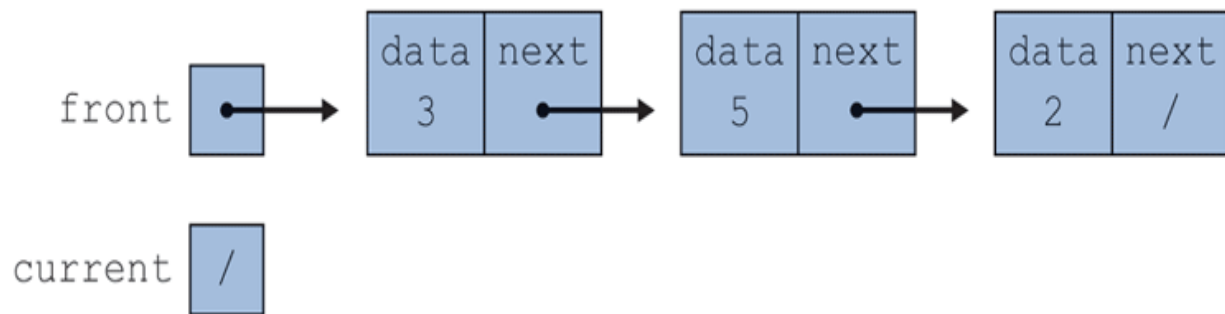
Next let's consider how to write a method that will append a value to the end of the list. To do so, we have to locate the end of the list. Let's think about the general case in which we are appending a value to the end of a list that already contains a few values. For example, suppose the list stores `[3, 5, 2]`:



Suppose that you want to add the value `17` at the end of the list. First you have to get to the correct location. So here's a start:

```
ListNode current = front;  
while (current != null) {  
    current = current.next;  
}
```

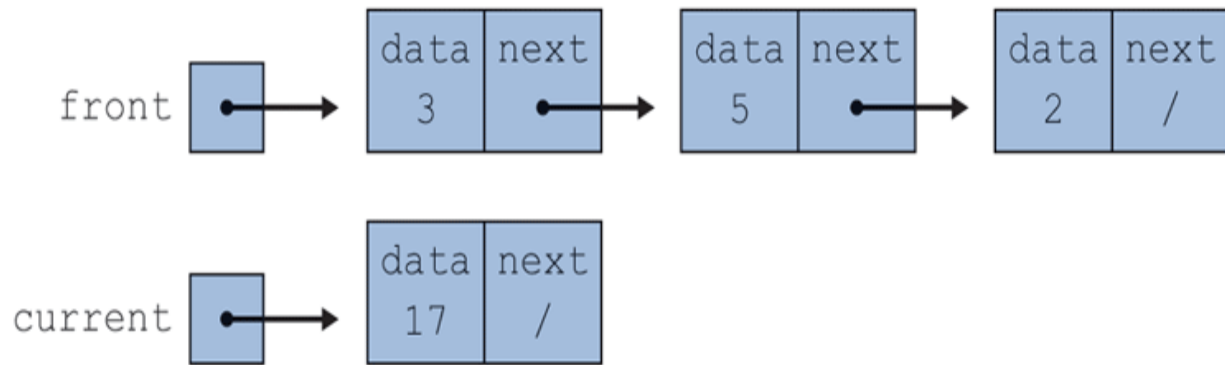
What happens when this code executes is that the variable `current` moves along the list from the first to the last node until the loop test fails—that is, until `current` becomes `null`. The computer's memory then looks like this:



It's tempting to think that you could then execute the following line of code to complete the task:

```
current = new ListNode(17);
```

But that won't work. It leaves the computer's memory looking like this:



The preceding code allocates a new node, but this new node has no connection to the original list. The list is still composed of three nodes linked together. This fourth node has been constructed, but it hasn't been properly linked into the list.

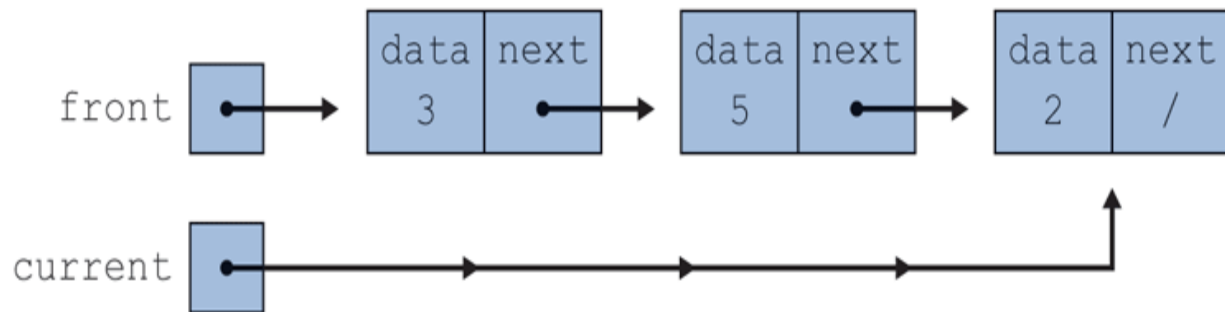
As you learn about linked list programming, you'll find that there are only two ways to change the contents of a list:

- change the value of `front`, in which case you are changing the starting point for the list, or
- change the value of `<variable>.next` (for some variable), which changes one of the current links of the list.

To solve this problem, you have to stop one position early. You don't want to run off the end of the list as you did with the `print` and `toString` code. Instead, you want to position `current` to the final element. You can do this by changing your test. Instead of running the loop until `current` becomes `null`, you want to run it until `current.next` is `null`, because only the last node of the list will have a `next` field that is `null`:

```
ListNode current = front;  
while (current.next != null) {  
    current = current.next;  
}
```

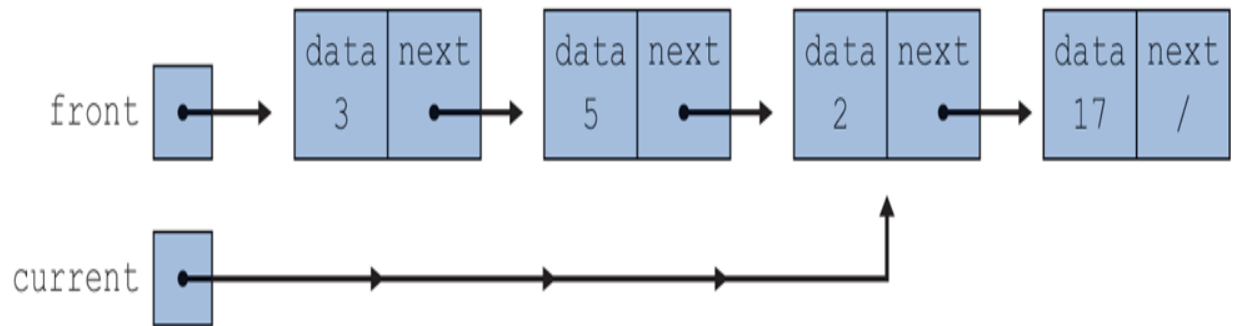
After the loop executes, `current` will be pointing at the last node in the list:



At that point, you can assign `current.next` to be a new node with `17` in it:

```
current.next = new ListNode(17);
```

In this case, you are changing the currently `null` value in that last node to instead link to a new node with `17` in it:



You have been preparing to write code for the appending `add`. So this code would be included inside a method, and you would have to alter it to use the name of the parameter:

```
public void add(int value) {  
    ListNode current = front;  
    while (current.next != null) {  
        current = current.next;  
    }  
    current.next = new ListNode(value);  
}
```

Even this code isn't quite correct, because you have to deal with the special case in which the list is empty:

```
public void add(int value) {  
    if (front == null) {
```

```

        front = new ListNode(value);
    } else {
        ListNode current = front;
        while (current.next != null) {
            current = current.next;
        }
        current.next = new ListNode(value);
    }
}

```

If you combine the `toString` method and the `add` method, along with a constructor that initializes the front of the list to `null`, you end up with the following first version of the `LinkedList` class:

```

1 // Simple first version of LinkedList with just a
  constructor
2 // and methods for add and toString.
3
4 public class LinkedList {
5     private ListNode front; // first value in the list
6
7     // post: constructs an empty list
8     public LinkedList() {
9         front = null;
10    }
11

```

```
12      // post: returns comma-separated, bracketed version of
list
13      public String toString() {
14          if (front == null) {
15              return "[]";
16          } else {
17              String result = "[" + front.data;
18              ListNode current = front.next;
19              while (current != null) {
20                  result += ", " + current.data;
21                  current = current.next;
22              }
23              result += "]";
24              return result;
25          }
26      }
27
28      // post: appends the given value to the end of the list
29      public void add(int value) {
30          if (front == null) {
31              front = new ListNode(value);
32          } else {
33              ListNode current = front;
34              while (current.next != null) {
35                  current = current.next;
36              }
37              current.next = new ListNode(value);
38          }
```

```
39     }  
40 }
```

The idea of stopping one step early in processing a list isn't limited to this appending operation. It is a fundamental pattern that comes up over and over in linked list programming, as you'll see with the other examples in the chapter.

The Middle of the List

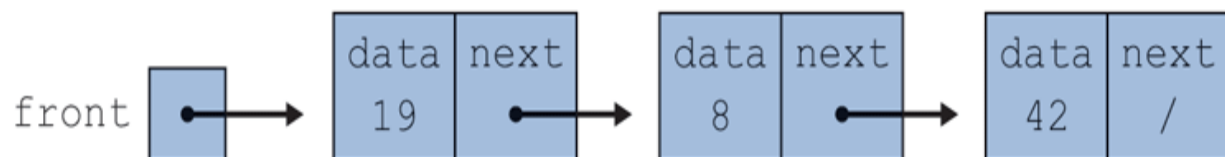
If you want your `LinkedList` to have the same capabilities as the `ArrayIntList` that we developed in [Chapter 15](#), you have to add several new methods. For example, you will want to have a method called `get` that returns a value at a given index. When the underlying structure was an array, you could simply ask for the array element at that index. But for a linked list, you have access only to the front of the list. So, to find a value at a particular index, you have no choice but to start at the front and examine each value until you get to the desired location:

```
public int get(int index) {  
    ListNode current = front;  
    for (int i = 0; i < index; i++) {  
        current = current.next;  
    }  
}
```

```
}  
  
return current.data;  
  
}
```

As we noted earlier, this means that your linked list does not have the fast random access property that arrays and our `ArrayIntList` have. Instead, we say that the linked list has sequential access, meaning that you have to go through the values in sequence until you get to the value of interest.

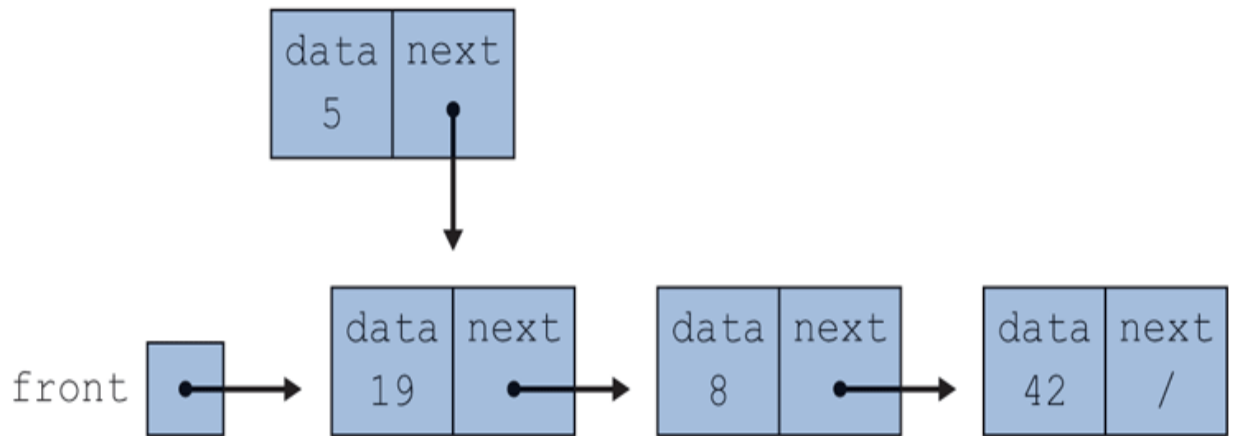
Let's now consider the task of adding a value at a particular index. There is a special case when you add at index 0, at the front of the list. For example, suppose a list contains the values `[19, 8, 42]`:



Suppose you want to add the value `5` at the front of this list. Then you will need to construct a new node that has the value `5` and that points to the current front of the list:

```
new ListNode(5, front)
```

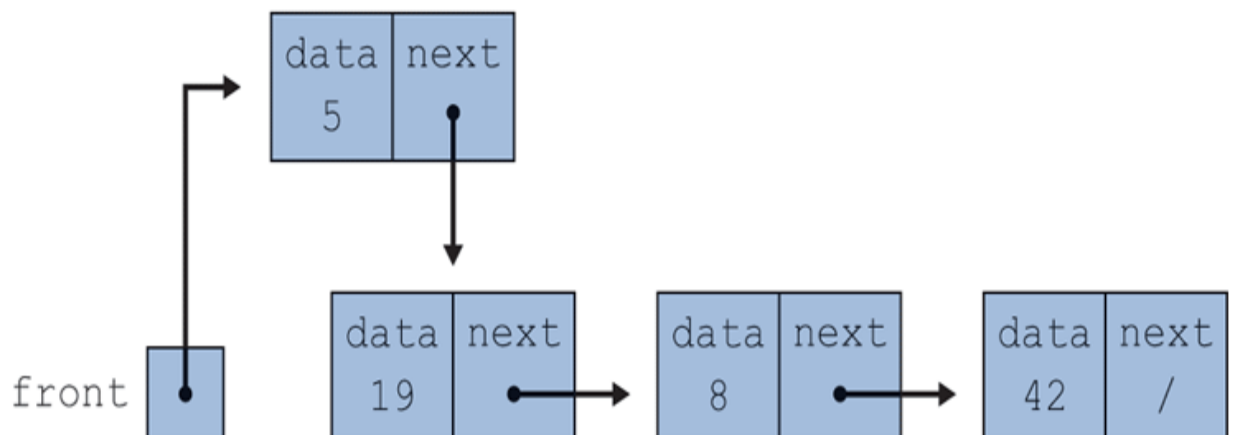
Constructing this node generates the following situation:



But that's only part of what you need to do. To link this node into the list, you have to reset `front` to point to this new node:

```
front = new ListNode(5, front);
```

Executing this line of code generates the following list:



This is a special case for your `add` method that applies only when the index is `0`. So your method to add at an index should begin like this:

```
public void add(int index, int value) {  
    if (index == 0) {  
        front = new ListNode(value, front);  
    } else {  
        ...  
    }  
}
```

If you aren't adding at the front of the list, then you have to use a temporary variable to position yourself to the appropriate spot in the list. This is another case in which you have to stop one step early. In the case of the `get` method, you wanted to position a local variable to the node with the given index. Here you want to stop one step before that index:

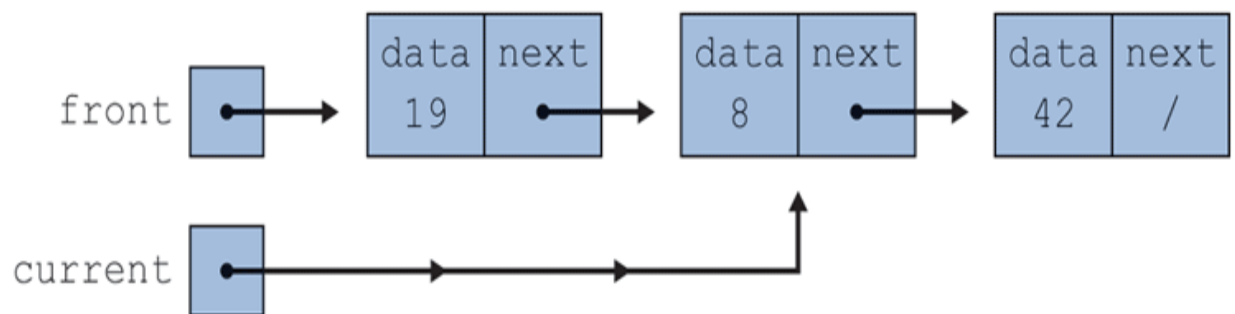
```
public void add(int index, int value) {  
    if (index == 0) {  
        front = new ListNode(value, front);  
    } else {  
        ListNode current = front;
```

```

    for (int i = 0; i < index - 1; i++) {
        current = current.next;
    }
    ...
}
}

```

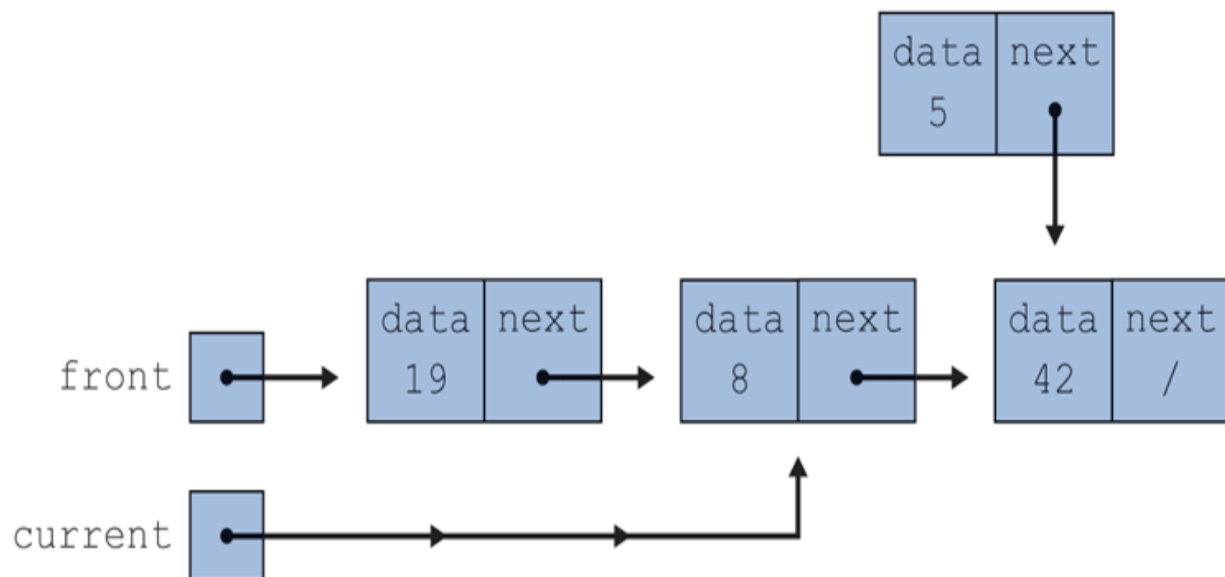
Notice that the `for` loop test uses `index - 1` instead of `index` so that it stops one step early. For example, suppose that you had the same starting list of `[19, 8, 42]` and that you want to insert the value `5` at index 2. You'd want to position the variable `current` to point at the value with index 1 (the node storing `8`):



Now you can change the value of `current.next` to add the new node. This new node should have a data value of `5`. To what should its `next` link refer? It should refer to the node that has `42` in it, which is stored in `current.next`. So you'll want to construct the node as follows:


```
new ListNode(5, current.next)
```

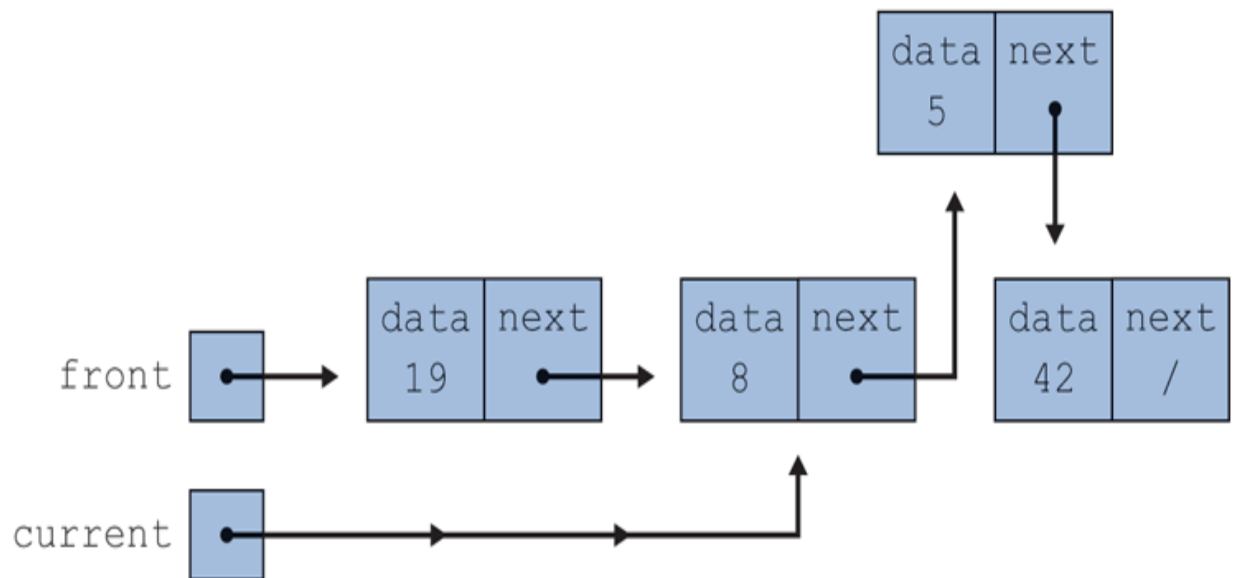
Just calling the constructor leaves the computer's memory looking like this:



Your code is not complete. You've constructed a node that points at the list, but nothing in the list points at the node. Thus, you've taken care of half of what you need to do. The other half of the task is to change a link of the list to point to the new node. The link to change is `current.next`:

```
current.next = new ListNode(5, current.next);
```

After this modification has been made, the list will look like this:



Some people prefer to write the code for creating the new node in two steps with a temporary variable, as in the following lines of code:

```
// first construct the node
ListNode temp = new ListNode(5, current.next);

// then link it into the list
current.next = temp;
```

Both approaches work. After you have incorporated this code into your method, the code looks like this:

```

public void add(int index, int value) {
    if (index == 0) {
        front = new ListNode(value, front);
    } else {
        ListNode current = front;
        for (int i = 0; i < index - 1; i++) {
            current = current.next;
        }
        current.next = new ListNode(value, current.next);
    }
}

```

You can use a similar approach to writing the `remove` method. You need to include a special case for the front of the list, in which case you can simply leapfrog over the first element:

```

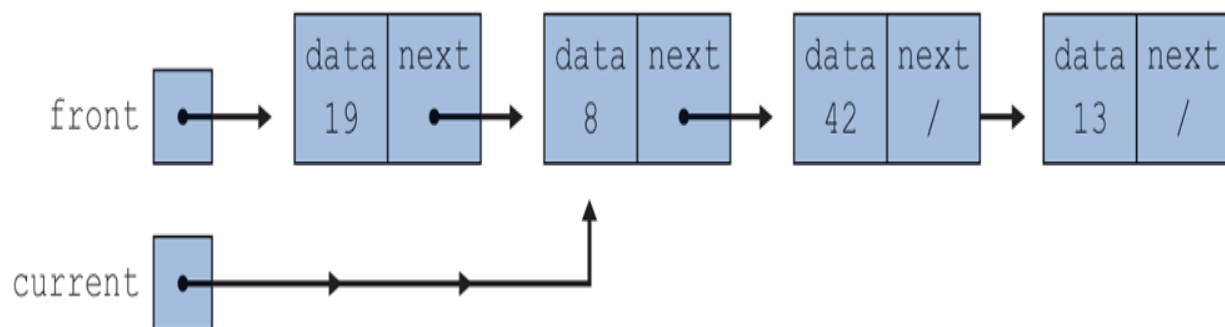
public void remove(int index) {
    if (index == 0) {
        front = front.next;
    } else {
        ...
    }
}

```

For values that appear later in the list, you need a loop similar to the one that you used in the `add` method to reach the position at the node just before the one to be removed:

```
public void remove(int index) {  
    if (index == 0) {  
        front = front.next;  
    } else {  
        ListNode current = front;  
        for (int i = 0; i < index - 1; i++) {  
            current = current.next;  
        }  
        ...  
    }  
}
```

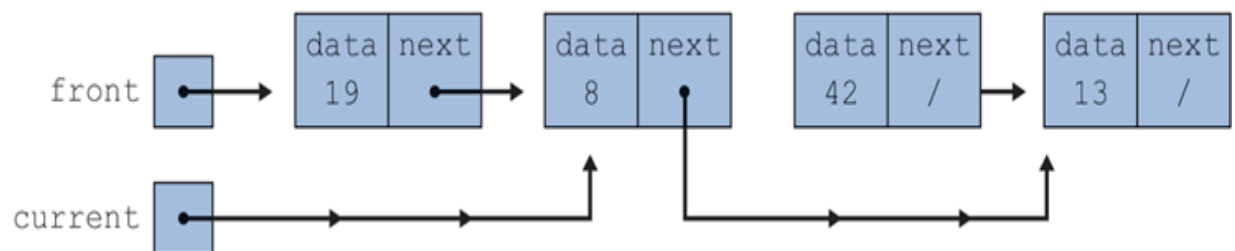
If you are working with the list `[19, 8, 42, 13]` and you are removing the value from index 2, your list will look like the following:



To remove the value from index 2, you need to change `current.next` to point to the value that comes next, which is stored in `current.next.next`:

```
public void remove(int index) {  
    if (index == 0) {  
        front = front.next;  
    } else {  
        ListNode current = front;  
        for (int i = 0; i < index - 1; i++) {  
            current = current.next;  
        }  
        current.next = current.next.next;  
    }  
}
```

Now the linked list looks like this:



The code has properly linked around the node that stores the value `42` (which is the value that was stored at index 2), so the list now

stores the sequence of values `[19, 8, 13]`.

As you write methods that involve manipulating values in the middle of the list, it becomes clear that you often end up writing the same kind of loop to position to a particular node in the list. This is a great place to introduce a private method:

```
private ListNode nodeAt(int index) {  
    ListNode current = front;  
    for (int i = 0; i < index; i++) {  
        current = current.next;  
    }  
    return current;  
}
```

The following is a complete implementation of the `LinkedList` that includes the methods we have just developed, along with the other basic operations that we included in our first version of `ArrayIntList`. Notice that the private `nodeAt` method is used in three different methods (`get`, `add` at an index, and `remove`). Here is the code:

```
1 // Class LinkedList can be used to store a list of  
  integers.  
2
```

```

3  public class LinkedIntList {
4      private ListNode front; // first value in the list
5
6      // post: constructs an empty list
7      public LinkedIntList() {
8          front = null;
9      }
10
11     // post: returns the current number of elements in the
list
12     public int size() {
13         int count = 0;
14         ListNode current = front;
15         while (current != null) {
16             current = current.next;
17             count++;
18         }
19         return count;
20     }
21
22     // pre : 0 <= index < size()
23     // post: returns the integer at the given index in the
list
24     public int get(int index) {
25         return nodeAt(index).data;
26     }
27
28     // post: returns comma-separated, bracketed version of

```

```

list
29     public String toString() {
30         if (front == null) {
31             return "[]";
32         } else {
33             String result = "[" + front.data;
34             ListNode current = front.next;
35             while (current != null) {
36                 result += ", " + current.data;
37                 current = current.next;
38             }
39             result += "]";
40             return result;
41         }
42     }
43
44     // post: returns the position of the first occurrence of
the
45     //         given value (-1 if not found)
46     public int indexOf(int value) {
47         int index = 0;
48         ListNode current = front;
49         while (current != null) {
50             if (current.data == value) {
51                 return index;
52             }
53             index++;
54             current = current.next;

```



```
55     }
56     return -1;
57 }
58
59 // post: appends the given value to the end of the list
60 public void add(int value) {
61     if (front == null) {
62         front = new ListNode(value);
63     } else {
64         ListNode current = front;
65         while (current.next != null) {
66             current = current.next;
67         }
68         current.next = new ListNode(value);
69     }
70 }
71
72 // pre: 0 <= index <= size()
73 // post: inserts the given value at the given index
74 public void add(int index, int value) {
75     if (index == 0) {
76         front = new ListNode(value, front);
77     } else {
78         ListNode current = nodeAt(index - 1);
79         current.next = new ListNode(value,
current.next);
80     }
81 }
```

```
82
83     // pre : 0 <= index < size()
84     // post: removes value at the given index
85     public void remove(int index) {
86         if (index == 0) {
87             front = front.next;
88         } else {
89             ListNode current = nodeAt(index - 1);
90             current.next = current.next.next;
91         }
92     }
93
94     // pre : 0 <= i < size()
95     // post: returns a reference to the node at the given
index
96     private ListNode nodeAt(int index) {
97         ListNode current = front;
98         for (int i = 0; i < index; i++) {
99             current = current.next;
100         }
101         return current;
102     }
103 }
```

16.3 A Complex List Operation

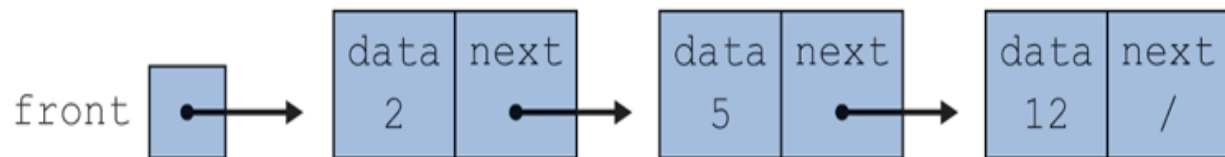


Suppose that you want to write a new method for your `LinkedList` class called `addSorted` that would add values to the list in a way which preserves sorted order. In other words, the method would be specified as follows:

```
// pre : list is in sorted (nondecreasing) order
// post: given value is added to the list so as to preserve
sorted
//      (nondecreasing) order, duplicates allowed
public void addSorted(int value) {
    ...
}
```

Our exploration of this task will point out important cases to consider and significant pitfalls that you must be careful to avoid when you program with linked lists.

Let's first look at the general case of adding a value to the middle of a list. Suppose that the list currently stores the values `[2, 5, 12]`:



Now suppose that you call the `addSorted` method and pass it the value `10`. How do you add the value `10` to this list to preserve sorted order? First you need to find the right spot to insert it. It belongs between the `5` and `12`, because it is larger than `5` and smaller than `12`. And how do you write that as a loop? You have to compare the value against the various data values stored in the list, starting with the first value. The new node doesn't belong in front of the node with `2` in it because `2` is less than `10`. Likewise, it doesn't belong in front of the node with `5` in it because `5` is less than `10`. But it does belong in front of the node with `12` in it, because `12` is not less than `10`. Your first attempt at writing the code might look like the following:

```
ListNode current = front;
while (current.data < value) {
```

```
    current = current.next;
}
```

This code has the core of the right idea, but it has many problems. First of all, it ends up positioning you in the wrong spot. You want to stop one position early in order to add something to the list, just as you did with the two `add` methods. Remember that there are only two ways to change the structure of a linked list: Either change `front`, or change one of the `next` fields of one of the nodes.

In this case, you want to change the `next` field of the node that has `5` in it. So you don't want your variable `current` to end up referring to the node that has `12` in it. You want `current` to point to the node that has `5` in it. You have to modify the code to stop one position early. You can do this by changing the test to use `current.next` instead of `current`:

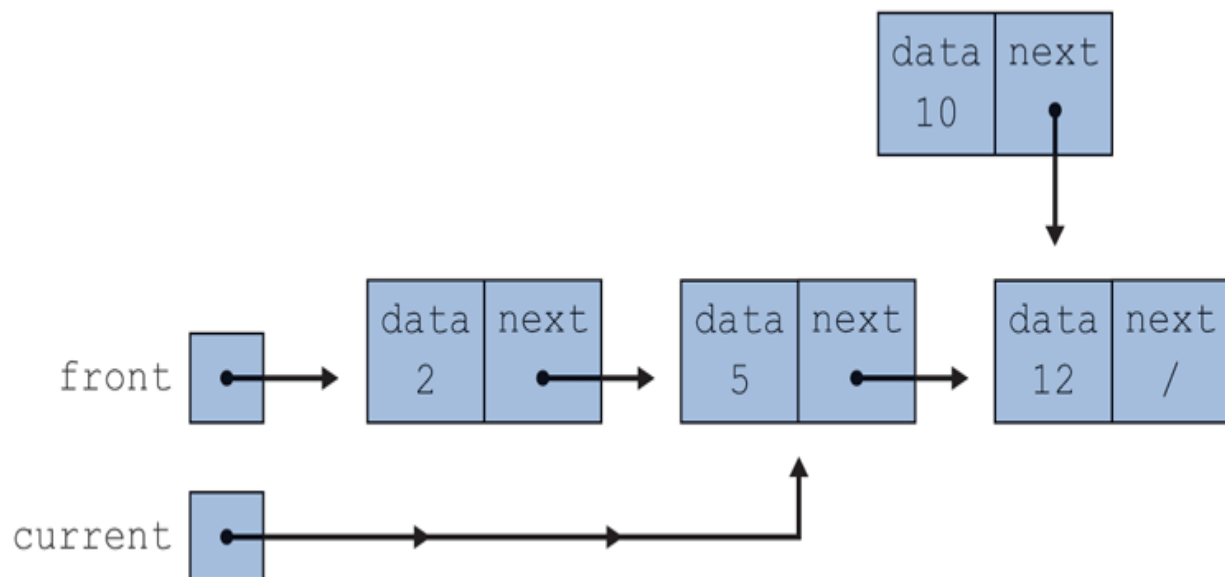
```
ListNode current = front;
while (current.next.data < value) {
    current = current.next;
}
```

In effect, your test now says, “While the value of the data one position to the right of my current position is less than `value`, keep

advancing `current`.” This loop stops when `current` refers to the node with `5` in it, which means that you can link in the new node by changing `current.next`. This new node should have a data value of `value` (`10` in our example). To what should its next link refer? It should refer to the node that has `12` in it, which is stored in `current.next`. So you’ll want to construct the node in the following way:

```
new ListNode(value, current.next)
```

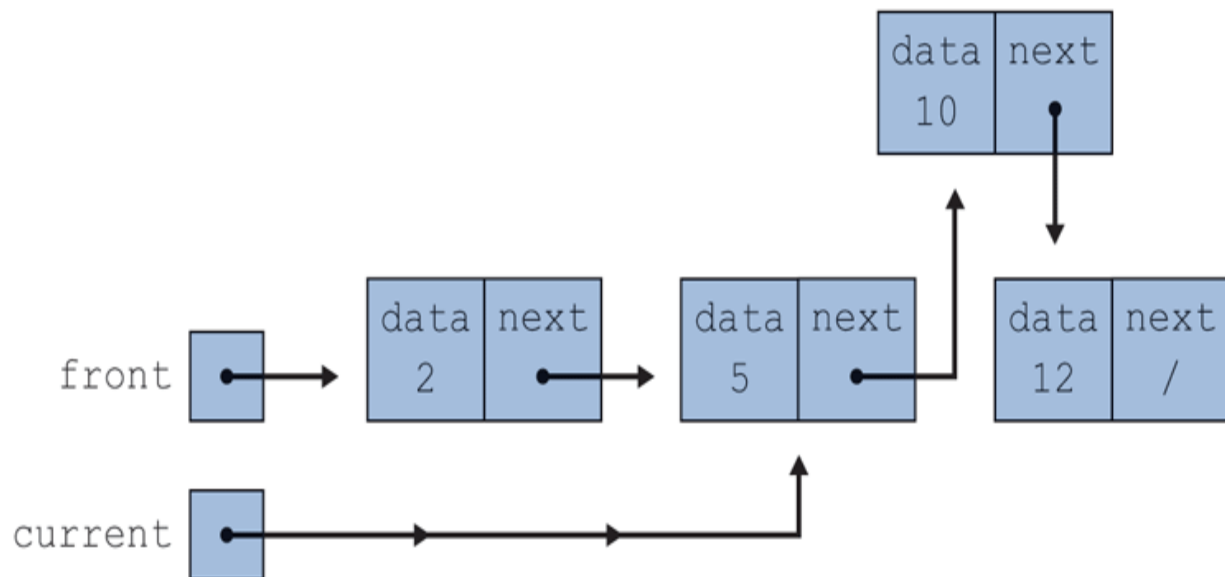
Just calling the constructor leaves the computer’s memory looking like this:



But now you have to link this into the list by changing `current.next`:

```
current.next = new ListNode(value, current.next);
```

Now the computer's memory looks like this:



This diagram isn't the easiest picture to read, but if you follow the links carefully, you'll see that, starting at the front, the sequence of values is 2, 5, 10, 12, which is what you want.

The code works for most cases, but it is incomplete. What if you want to insert the value 42? Remember your loop test:

```
while (current.next.data < value)
```

This code depends on finding a value in the list that is less than the value you are trying to insert. What if there is no such value, as in the case of 42? This code keeps moving `current` forward until `current.next` is `null`. At that point, when you try to ask for the value of `current.next.data`, you are asking for `null.data`, which throws a `NullPointerException` because Java is looking for the `data` field of a nonexistent object.

If the value that you wish to insert is greater than everything else in the list, then it belongs after the last node in the list. So you want to stop when `current` gets to the last node in the list. Thus, a second (still incorrect) attempt at writing the code for the test would be

```
while (current.next.data < value && current.next != null)
```

This code still doesn't work. The test for `current.next` being `null` should stop changing `current` at the right place, but when `current.next` is `null`, it is not legal to ask for the value of `current.next.data`. That test will throw a `NullPointerException`. This test is an example of a combination of a sensitive and robust test:

```
while (current.next.data < value && current.next != null)
```



sensitive test robust test

You need to switch the order of these tests to make them work properly.

```
while (current.next != null && current.next.data < value)
```

Recall from [Chapter 5](#) that Java uses what is known as short-circuited evaluation, which means that if the first test evaluates to `false`, Java doesn't bother to perform the second test. So the first test, in effect, protects you from the potential problem generated by the second test (the `NullPointerException`).

Incorporating these changes, you end up with the following code:

```
ListNode current = front;
while (current.next != null && current.next.data < value) {
    current = current.next;
}
current.next = new ListNode(value, current.next);
```

But even this code is not enough. The first test in this loop is the robust test, but it isn't very robust. If `current` is `null`, then it throws a `NullPointerException`. So you want to execute this code only in the case in which `front` isn't `null`.

There is another special case. If the value to be inserted belongs at the very front of the list, then this code will place it in the wrong spot. It always inserts after a node currently in the list, never in front of all nodes.

For example, suppose that you want to insert the value `1` in the previous list that begins with the value `2`. The code that you have written starts `current` at the front of the list and inserts the value *after* that node by changing the value of `current.next`. So the value `1` would be inserted after the value `2`, which is clearly wrong.

For the “front of the list” case, you have to write code that changes `front` rather than changing `current.next`. In what case would you want to do that? When the value is less than `front.data`. And what exactly do you want to do? You want to set `front` to a new list node that points at the old front of the list:

```
if (value < front.data) {  
    front = new ListNode(value, front);  
}
```

There is yet another special case. If you ran this program, you’d find that, after all of your hard work, the program throws a `NullPointerException` on the very first call to `addSorted`. Java throws

an exception if the list is empty because a line in the code asks for `front.data` and `front` is `null`.

So you need to include yet another test in the code:

```
if (value <= front.data || front == null) {  
    front = new ListNode(value, front);  
}
```

Of course, we have purposely written this addition in the wrong way as well. This is another example of a sensitive test (referring to `front.data`) and a robust test (testing `front` for `null`). So you have to reverse the order of these two tests to make the modification to the code work properly.

The final version of the method is as follows:

```
public void addSorted(int value) {  
    if (front == null || front.data >= value) {  
        front = new ListNode(value, front);  
    } else {  
        ListNode current = front;  
        while (current.next != null && current.next.data <  
value) {
```

```
        current = current.next;
    }

    current.next = new ListNode(value, current.next);
}
}
```

The `if` statement in this code deals with the two special cases we just mentioned. If the list is currently empty (`front == null`) or if the value belongs at the front of the list (`front.data >= value`), then you insert the value at the front of the list rather than using the other code that you developed. The order in which this test appears is important because the test involving `front.data` will throw a `NullPointerException` if `front` is `null`.

This code is a good example to study because it has so many special cases. In the course of writing the code, you had to deal with the following cases:

- **middle:** the “typical” case in which a value is inserted in the middle of the list
- **back:** the special case in which we had to insert at the back of the list
- **front:** the special case in which we had to insert at the front of the list
- **empty:** the special case in which we had to insert into an empty list

The first two cases are handled by the `if` branch of the code:

```
if (front == null || front.data >= value)
```



empty list **front of list**

The second two cases are handled inside the `else` branch of the code:

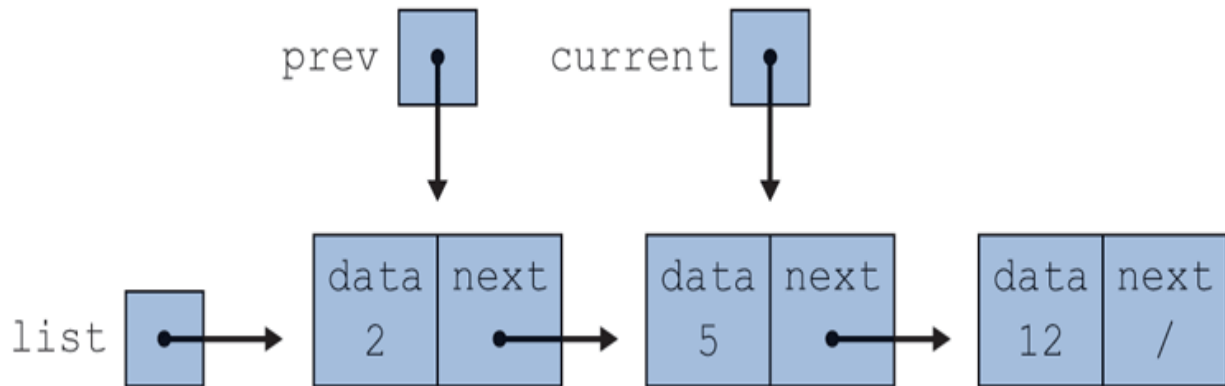
```
while (current.next != null && current.next.data < value)
```



back of list **middle of list**

Inchworm Approach

Some people find it confusing to write code that involves expressions like `current.next.data`. Another approach is to use a pair of pointers that keep track of a current and a previous position in the list. In effect, the pointers keep track of a two-element window on the list, as in the following diagram:



As an analogy, consider an inchworm that is two nodes in length. When the inchworm stretches out, its back half is on one node and the front half is on another node. To move forward, it scoots its back half up to where the front half is, then scoots the front half onto the next node. This is exactly analogous to the code you'd use to move this pair of variables forward one spot:

```
prev = current;  
current = current.next;
```

Here is a solution to the `addSorted` problem using this approach:

```
public void addSorted(int value) {  
    if (front == null || front.data >= value) {  
        front = new ListNode(value, front);  
    } else {
```

```

        ListNode prev = front;
        ListNode current = front.next;
        while (current != null && current.data < value) {
            prev = current;
            current = current.next;
        }
        prev.next = new ListNode(value, prev.next);
    }
}

```

Yet another variation is to set `prev` equal to `null` initially in order to eliminate the special case for the front of the list. Then you have to test after the loop to see whether `prev` is still `null`:

```

public void addSorted(int value) {
    ListNode prev = null;
    ListNode current = front;
    while (current != null && current.data < value) {
        prev = current;
        current = current.next;
    }
    if (prev == null) {
        front = new ListNode(value, front);
    } else {
        prev.next = new ListNode(value, prev.next);
    }
}

```

```
}
```

```
}
```


16.4 An `IntList` Interface

The `ArrayIntList` class from [Chapter 15](#) and the `LinkedList` class we have explored in this chapter have very similar methods. They each have the following methods:

- a `size` method
- a `get` method
- a `toString` method
- an `indexOf` method
- a one-argument `add` method (the appending `add`)
- a two-argument `add` method (add at an index)
- a `remove` method (remove at an index)

This similarity isn't an accident. When we began studying linked lists, we purposely implemented new versions of these methods that worked for linked lists. The point is that these classes are similar in terms of what they can do, but they are very different in the ways that they do it.

Consider the following client code, which performs parallel operations on two different lists, adding three values, removing one, and printing the list before and after the `remove`:

```
1 // A client program that uses two list classes.
2 public class ListClient1 {
3     public static void main(String[] args) {
4         ArrayIntList list1 = new ArrayIntList();
5         list1.add(18);
6         list1.add(27);
7         list1.add(93);
8         System.out.println(list1);
9         list1.remove(1);
10        System.out.println(list1);
11
12        LinkedIntList list2 = new LinkedIntList();
13        list2.add(18);
14        list2.add(27);
15        list2.add(93);
16        System.out.println(list2);
17        list2.remove(1);
18        System.out.println(list2);
19    }
20 }
```

The program produces the following output:

```
[18, 27, 93]
```

```
[18, 93]
[18, 27, 93]
[18, 93]
```

As we expected, the two kinds of list behave the same way. So what if you wanted to introduce a static method to eliminate this redundancy? What type would you use for the parameter? You can't use `ArrayList` because then you wouldn't be able to pass the `LinkedList` as a parameter, and vice versa.

As we saw in [Chapters 9](#) and [11](#), the right way to approach this task is to introduce an interface. You want to think of these lists as being the same kind of thing. Even though you recognize that there are significant differences between the two, you can imagine an “integer list” abstraction of which these are two possible implementations. They're the same in the sense that they provide basic “integer list” functionality like an appending `add`. But they are different in the sense that they are implemented quite differently (one using an array and the other using a linked list).

You can introduce an interface with just the method headers to capture this notion of the integer list:

```
1 // Interface for a list of integers.
2 public interface IntList {
3     public int size();
```

```
4     public int get(int index);
5     public String toString();
6     public int indexOf(int value);
7     public void add(int value);
8     public void add(int index, int value);
9     public void remove(int index);
10 }
```

Using the interface, you can write a `processList` method to use in the client program:

```
1 // A client program that uses lists via an interface.
2 public class ListClient2 {
3     public static void main(String[] args) {
4         ArrayIntList list1 = new ArrayIntList();
5         processList(list1);
6
7         LinkedIntList list2 = new LinkedIntList();
8         processList(list2);
9     }
10
11     public static void processList(IntList list) {
12         list.add(18);
13         list.add(27);
14         list.add(93);
15         System.out.println(list);
16     }
17 }
```

```
16         list.remove(1);  
17         System.out.println(list);  
18     }  
19 }
```

If this is the only change you make, then your program will include two errors instead of one. Now both calls on the method cause an error. That seems a bit odd, because both `ArrayIntList` and `LinkedList` have the methods mentioned in the `IntList` interface. But recall that Java requires classes to explicitly state which interfaces they implement. So you have to modify the two classes to include this notation:

```
public class ArrayIntList implements IntList {  
    ...  
}  
  
public class LinkedList implements IntList {  
    ...  
}
```

After you make this change, the code compiles and executes properly.

It is important to keep in mind that you can't create instances of the interface type. The following line of code is an error:

```
IntList list = new IntList(); // error-can't instantiate the
interface
```

Interfaces cannot be instantiated because they are incomplete. They specify behaviors, but they don't say how those behaviors are implemented. So when you create an instance, you have to pick one of the actual implementations.

Even though you can't create an instance of an interface, it is a good idea to use interfaces when you define the types of variables.

Therefore, method `main` in the client code should be rewritten as follows:

```
public static void main(String[] args) {
    IntList list1 = new ArrayIntList();
    processList(list1);

    IntList list2 = new LinkedIntList();
    processList(list2);
}
```

These variables are more flexible than the old variables. The variable `list1`, for example, can now refer to any `IntList` object, not

just one of type `ArrayList`.

16.5 LinkedList<E>

In this section, we will use the `LinkedList` that we have developed to produce a class that is similar to Java's `LinkedList<E>` class. In the collections framework, there is a close relationship between the `LinkedList<E>` class and the `ArrayList<E>` class. We'll mirror that relationship in the version that we create. The task will require making a few changes to the `ArrayList<E>` that we developed at the end of the previous chapter.

In particular, we'll introduce an interface `List<E>` that both classes will implement. We can include all of the methods that we implemented for the `ArrayList<E>`.

We can also introduce a slight improvement. Various structures can use the for-each loop that was introduced in [Chapter 7](#) . If you want to be able to use a for-each loop for your own class, you have to implement a special interface known as `Iterable<E>`. It has one and only one method:

```
public interface Iterable<E> {  
    public Iterator<E> iterator();  
}
```

In other words, we need to have a method that constructs an iterator for our collection. We'll discuss later in this section how to do that for a linked list. Under the hood, the for-each loop actually calls the `iterator` method on the collection and yields each next element to the loop's body.

In this case, we want our two classes to implement both the `List<E>` interface and the `Iterable<E>` interface. The usual way to accomplish that in Java is to have one interface extend another. This is similar to inheritance for classes and you use the same `extends` keyword.

Here is the complete `List<E>` specification:

```
1 // Generic interface for a List of objects of type E.
2
3 public interface List<E> extends Iterable<E> {
4     public int size();
5     public E get(int index);
6     public int indexOf(E value);
7     public boolean isEmpty();
8     public boolean contains(E value);
9     public void add(E value);
10    public void add(int index, E value);
11    public void addAll(List<E> other);
12    public void remove(int index);
13    public void set(int index, E value);
```

```
14     public void clear();  
15 }
```

Notice that the header mentions that it extends the `Iterable<E>` interface. That means that any class implementing the interface must provide a method for constructing an iterator, which will allow the class to be used in for-each loops.

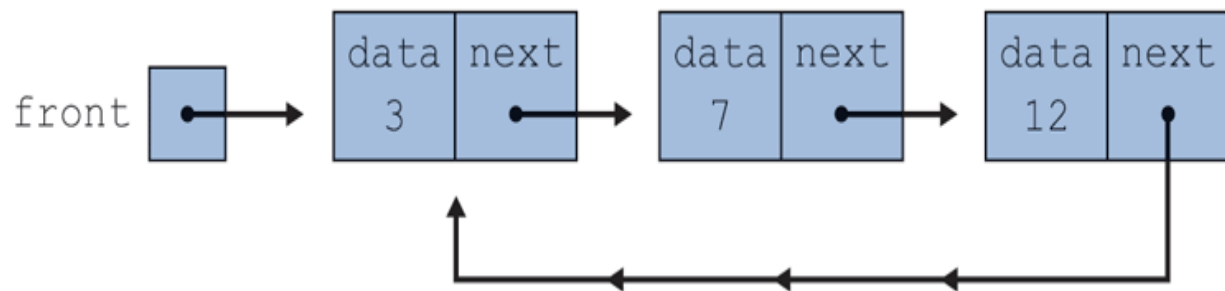
Before we write the actual code for `LinkedList<E>`, we'll explore some variations on linked lists and discuss the issue of iterators.

Linked List Variations

So far in this chapter, we have discussed the simplest possible linked list. Each node in this list has had a single link to the next value in the list, and we have terminated each list with the value `null`. We have used the value `null` to represent an empty list.

However, there are many variations on linked lists. Several of these ideas are used in Java's implementation of `LinkedList`, and we will use several for our own implementation. There are three primary variations: circular lists, lists with dummy nodes, and doubly linked lists. Let's consider each variation in turn.

Instead of making the final element of the list store the value `null`, we can have the final element of the list point back to the first value in the list. We refer to such a list as *circular*. For example, here is a circular list that stores the values `[3, 7, 12]`:



There are many applications in which it can be helpful to have the list point back to the front in this manner. To manipulate such a list, you use very different code. You can't test for `null` because there are no `null` links in the list. Instead, you test whether you have returned to the point at which you started. It is often helpful to use a `do/while` loop for processing a circular list. For example, the following code prints the values of the list, one per line:

```
ListNode current = front;
do {
    System.out.println(current.data);
    current = current.next;
} while (current != front);
```

One of the tricky aspects of working with a circular list is dealing with an empty list. The preceding code, for example, generates a `NullPointerException` if the variable `list` has the value `null`.

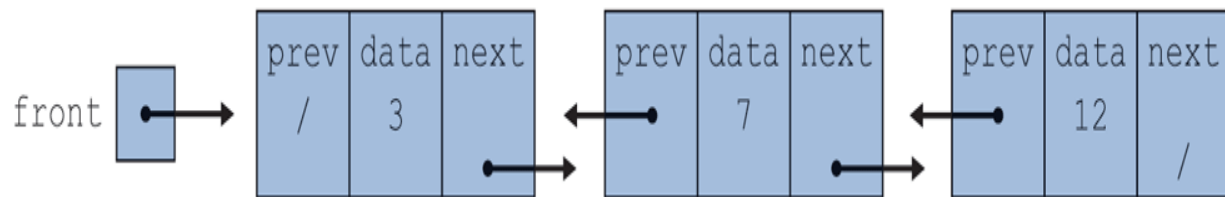
This brings us to our second variation. It is often helpful to have extra nodes in the list that do not store meaningful elements and that are not considered part of the list. We refer to such a node as a *dummy node*. For example, in a circular list it is often convenient to represent the empty list as a single dummy node that points back to itself. Dummy nodes are useful even with the kind of null-terminated lists we have been writing. By using a dummy header node, we represent the empty list as having just the dummy node:



The `data` field in the preceding list stores `0`, but it doesn't matter what value we store in the dummy node. The advantage of using a dummy header node is that we no longer have to write special code to insert a value at the front of the list. Now the front of the list will always be this dummy header node. For example, if we were to insert the values `3`, `7`, and `12`, they would be inserted after the dummy node:



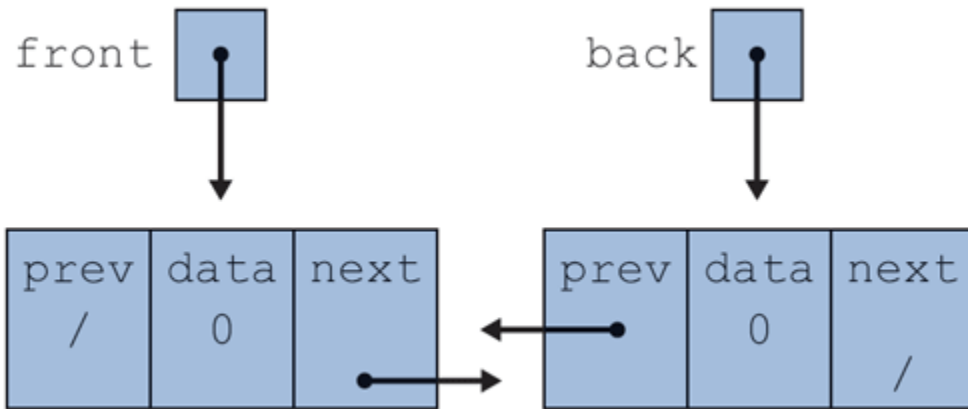
The final linked list variation is called a *doubly linked list*. The idea is that instead of storing links in a single direction, we store them in both directions. That means that our node class has fields for both `next` and `prev` and that we maintain the links in both directions. For example, the three-element list `[3, 7, 12]` would be stored as follows:



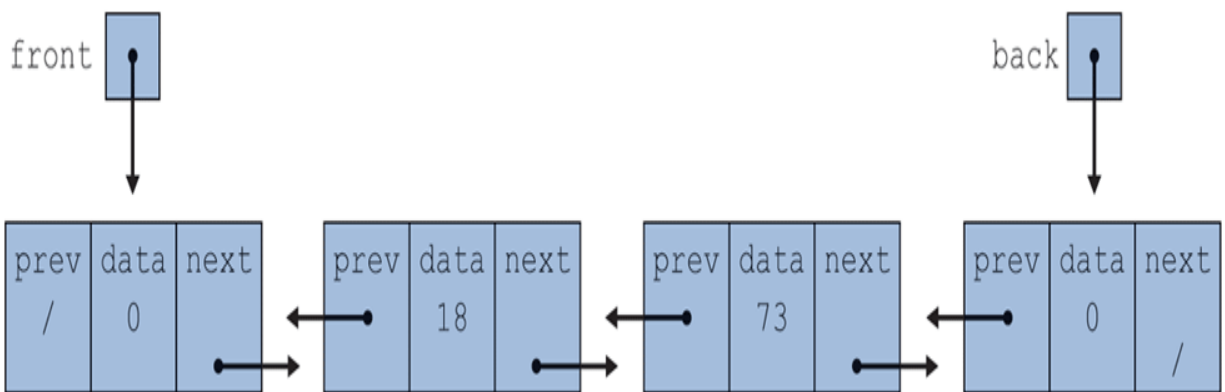
Notice that each node now has two different links pointing in different directions. The list also has two `null` links. The final node has a `next` field that is `null` and the first node has a `prev` field that is `null` to indicate in each case that there are no other nodes in that direction.

In practice, programmers use some combination of these techniques. For example, Java's `LinkedList<E>` is implemented as a doubly linked, circular list.

In our implementation, we will use two of these techniques. Our list will be doubly linked and will have two dummy nodes, one for the front of the list and one for the back of the list. We will also introduce an extra field to keep track of the back of the list. Thus, in our version, an empty list will look like this:



This list doesn't look very empty, but that's the idea. Even when it is empty, our list will have two dummy header nodes. That way, the fields `front` and `back` will always point to exactly the same dummy node and we won't have to write any special case code for changing their values. Instead, all insertions will occur between the two dummy nodes. For example, if we insert the values `18` and `73`, our list will look like this:



You can think of the dummy nodes as bookends that appear on either side of the actual nodes.

Linked List Iterators

To match our `ArrayList` class and to be able to implement the `Iterable` interface that allows us to use for-each loops, we need to implement an iterator for our `LinkedList` class. Just as we did with `ArrayList`, we can make the iterator class an inner class that has access to the fields of the outer object. In the case of the `ArrayList`, the primary field for our iterator was an integer index. For a linked list, we want to keep track of a current list node. We still need the `removeOK` field that we used for `ArrayList`. So our iterator class begins as follows:

```
private class LinkedIterator implements Iterator<E> {  
    private ListNode<E> current; // location of next value to  
return  
    private boolean removeOK; // whether it's okay to remove  
now  
    ...  
}
```

Notice that our node class is now also generic (`ListNode<E>`). As in the last chapter, we do not declare our class as `LinkedIterator<E>`, because the type `E` is already declared as part of the outer linked list

class. But we do place an `<E>` type parameter after the `Iterator` and `ListNode` declarations.

To implement the iterator class, you have to take into consideration the fact that you will have two dummy nodes, one at the front of the list and one at the back. The first node you will want to process will come after that dummy node at the front, so the proper initialization for your iterator is as follows:

```
public LinkedIterator() {  
    current = front.next;  
    removeOK = false;  
}
```

To test whether you have a next value to examine, you will use the dummy nodes again. You'll want to continue to move `current` forward until it reaches the dummy node at the end of the list. That means that you'll implement `hasNext` as follows:

```
public boolean hasNext() {  
    return current != back;  
}
```


For the other two operations, you can simply translate what you did with the array implementation into appropriate code for your linked list implementation. For the `next` method, your code will look like the following:

```
public E next() {  
    if (!hasNext()) {  
        throw new NoSuchElementException();  
    }  
    E result = current.data;  
    current = current.next;  
    removeOK = true;  
    return result;  
}
```

For the `remove` method, your code will look like this:

```
public void remove() {  
    if (!removeOK) {  
        throw new IllegalStateException();  
    }  
    ListNode<E> prev2 = current.prev.prev;  
    prev2.next = current;  
    current.prev = prev2;  
}
```

```
size--;  
removeOK = false;  
}
```

Notice that, to remove a node, you have to reset both a `next` and a `prev` pointer in the list because now you are working with a doubly linked list.

Before we leave the subject of iterators, you should consider briefly how to implement the `addAll` method for your `LinkedList` class. Keep in mind that its header will refer to your `List` interface:

```
public void addAll(List<E> other) {  
    ...  
}
```

You have to write this code in such a way that it can take either a `LinkedList` or an `ArrayList` as a parameter. That means that you have to use methods mentioned in the interface. One misguided approach would be to use a `for` loop and repeatedly call `get` to retrieve each element:

```
// inefficient implementation of addAll method
```

```
public void addAll(List<E> other) {  
    for (int i = 0; i < other.size(); i++) {  
        add(other.get(i));  
    }  
}
```

This code will work, but it will be extremely inefficient when `other` is a `LinkedList`. Each time you call the `get` method, you have to start at the front of the list and skip nodes until you get to the right spot. It is much better to use an iterator to solve this problem. You can make it particularly simple by using a for-each loop and letting Java manage the iterator for you. This technique greatly simplifies the code:

```
public void addAll(List<E> other) {  
    for (E value : other) {  
        add(value);  
    }  
}
```

You should use this same implementation for the method in the `ArrayList` class to make sure that it also runs efficiently for all types of lists. It turns out that several of the methods can be implemented using the same code for both the array list and linked list. A better

implementation would introduce an abstract list superclass to represent this similarity. This is left as a programming project idea.

Other Code Details

We are almost ready to put the pieces together. But first, let's consider the fields for our `LinkedList` class. We have already discussed keeping track of both the front and the back of the list. It is also useful to keep track of the current size in a field so that you don't have to traverse the list to find out its size. So there will be three fields:

```
public class LinkedList<E> implements List<E> {  
    private ListNode<E> front;    // first value in the list  
    private ListNode<E> back;    // last value in the list  
    private int size;            // current number of elements  
  
    ...  
}
```

To initialize the list, you want to construct the two dummy nodes and then set up everything to refer to an empty list. You also have to implement the `clear` method that is supposed to return to an empty list, so this is a good opportunity to write the code once in the `clear`

method and to have the constructor call it. The constructor will look like this:

```
public LinkedList() {  
    front = new ListNode<>(null);  
    back = new ListNode<>(null);  
    clear();  
}
```

Notice that you need to include `<>` when you construct the nodes. For the `clear` method, you simply make these two dummy nodes point to each other and reset the size:

```
public void clear() {  
    front.next = back;  
    back.prev = front;  
    size = 0;  
}
```

When you implement the `get` method, you can introduce an optimization. In the case of a singly linked list, you always start at the beginning of the list and move forward until you find the appropriate spot in the list. This works well when the value appears toward the

beginning of the list. What if the value appears near the end? Because your list is doubly linked, you can start at the end and work backward, following the `prev` links until you get to the appropriate spot. The best thing to do is to start from the front when the index is in the first half of the list and to start from the back when it is in the second half of the list. This optimization applies to any of the methods that involve going to a specific index (`remove`, `set`, `add`). All of these methods call the private `nodeAt` method, so the optimization appears in just one place in the code:

```
private ListNode<E> nodeAt(int index) {  
    ListNode<E> current;  
    if (index < size / 2) { // start from the front  
        current = front;  
        for (int i = 0; i < index + 1; i++) {  
            current = current.next;  
        }  
    } else { // start from the back  
        current = back;  
        for (int i = size; i >= index + 1; i--) {  
            current = current.prev;  
        }  
    }  
    return current;  
}
```

In our final implementation of `LinkedList`, we didn't include methods for `isEmpty`, `indexOf`, or `contains`. The `isEmpty` method can simply check whether the size is `0`, as you did for the `ArrayList`, and the `contains` method can check the result of calling `indexOf`.

To write `indexOf`, you loop over the list, looking for the value and returning its index if you find it. Because of your dummy nodes, the loop ends up looking slightly different than the other loops you have written:

```
ListNode<E> current = front.next;
while (current != back) {
    ...
}
```

You also have to be careful to call the `equals` method for comparisons.

The code involves ordinary list operations that you've seen throughout this chapter:

```
public int indexOf(E value) {
    int index = 0;
    ListNode<E> current = front.next;
```

```

while (current != back) {
    if (current.data.equals(value)) {
        return index;
    }
    index++;
    current = current.next;
}
return -1;
}

```

Many of the methods that you wrote for `LinkedList` can be used with only slight modifications for the `LinkedList`. Let's look at `remove` as an example. Because of the dummy nodes, you don't have to make a special case for an empty list. But because of the doubly linking, you have to reset two links (a `next` and a `prev`) when you remove the node:

```

public void remove(int index) {
    checkIndex(index);
    ListNode<E> current = gotoIndex(index - 1);
    current.next = current.next.next;
    current.next.prev = current;
    size--;
}

```


You'll want to make similar changes to the `add` method.

The `LinkedList` class includes one final improvement. Instead of keeping the node class as a separate class, the `LinkedList` class includes it as a nested class. In this case, the individual node objects don't need access to the outer object, so it is best to declare the class to be a static inner class. The details of this code are beyond the scope of this book, but the quick explanation is that providing access to the outer class takes some extra storage and because we're likely to construct thousands and perhaps even millions of node objects, we want to be more efficient about the space that they take up. Declaring the class to be static accomplishes that goal.

The complete code for the `LinkedList` class can be found at <http://buildingjavaprograms.com>. You will also find the `List` interface and an updated version of `ArrayList` there.

Chapter Summary

A linked list is a data structure that stores an ordered sequence of elements using a chain of objects called nodes. Each node refers to at least one other node in the list. The overall list object keeps references to a small number of these nodes, such as the front or back node.

A node has a `data` and `next` field. You can connect one node to another by assigning its `next` field to refer to the other node. It is possible to make chains of nodes of arbitrary length in this way.

The end of a chain of nodes, the `next` reference of the last node, is `null`.

When you are trying to rearrange one ordering of nodes into another, you must be careful about the order in which you change the references. If you change the ordering so that no variable or field refers to a given node, the node is lost and cannot be recovered.

You can traverse a linked list by creating a temporary node reference (called `current` in our examples) and looping through the overall list of nodes. Such a loop can stop when the program finds a particular value or encounters a `null` next node (the end of the list).

It is often desirable to stop one node before the relevant part of the list that you are searching for, because, in order to change the list, you often must change the `next` field of the prior node.

When you are performing complex list operations such as adding to a sorted list, it is important to think about all of the possible cases in which the method might be called. Is the list empty, does it have a single element, or does it have many elements? Will our element of interest be added at or removed from the beginning, middle, or end of the list?

When you create two tests separated by an `&&` or `||`, be sure to place the more “robust” test before the more “sensitive” test, especially if the robust test’s failure would cause the sensitive test to throw an exception.

One way to traverse a list is with two current references that are one node apart, also called the inchworm approach.

A list interface can be created to represent an abstract data type that is implemented by both array lists and linked lists. The list interface allows client code to treat both types of lists the same way.

A linked list iterator keeps a reference to its current position in the list. This class is often written as an inner class inside the overall list class so that the iterator has direct access to the list's nodes.

A generic linked list class can store objects of any type `E` rather than just integers.

Self-Check Problems

Section 16.1: Working with Nodes

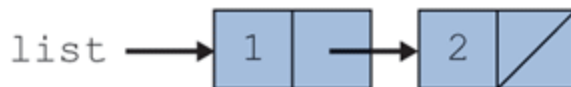
1. What is the difference between a linked list and an array list?
How are they similar?

2. What is the difference between a linked node and a linked list? How are they related and connected?

3. What value is stored as the `next` field of the last node of a list? What value will a node's `next` field have if none is specified?

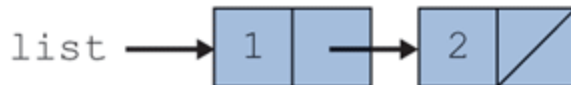
4. What happens if you or the client try to go past the last element in a linked list? Be specific.

For each of the next four problems, draw a picture of what the given linked nodes would look like after the given code executes.



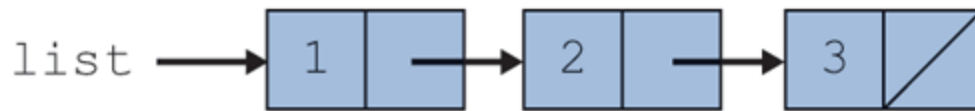
5.

```
list.next = new ListNode(3);
```



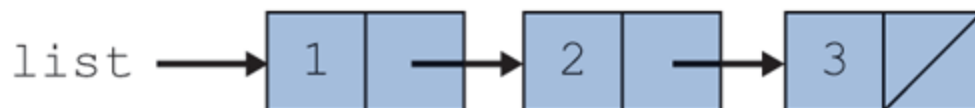
6.

```
list.next = new ListNode(3, list.next);
```



7.

```
list = new ListNode(4, list.next.next);
```



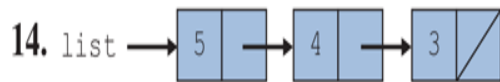
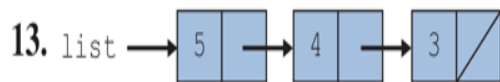
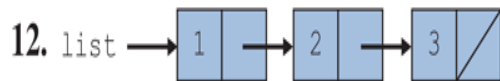
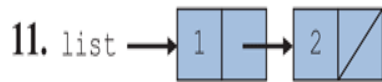
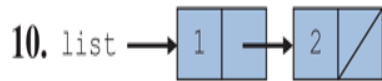
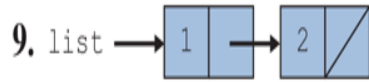
8.

```
list.next.next = null;
```

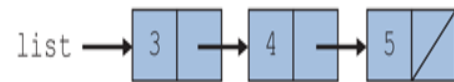
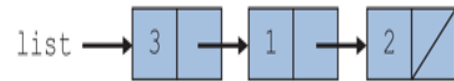
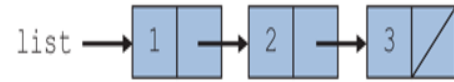
For each of the next nine problems, you'll see pictures of linked nodes before and after changes. Write the code that will produce the given result by modifying links between the nodes shown and/or creating new nodes as needed. There may be more than one way to write the code, but you may not change any existing node's `data` field value. If a variable does

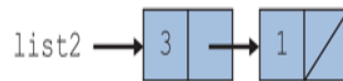
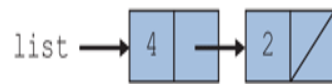
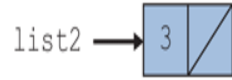
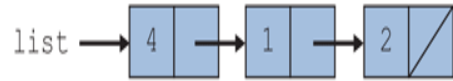
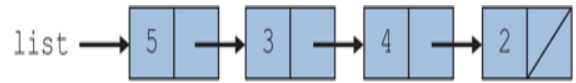
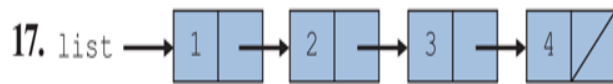
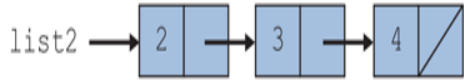
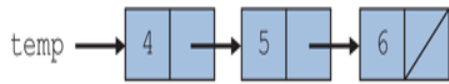
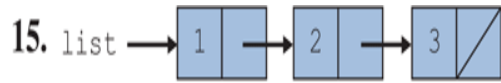
not appear in the “after” picture, it doesn’t matter what value it has after the changes are made.

Before



After

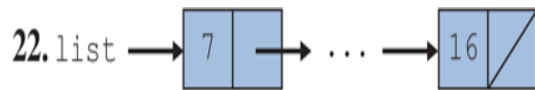




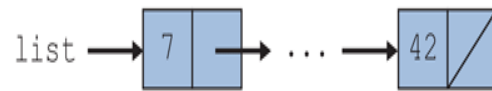
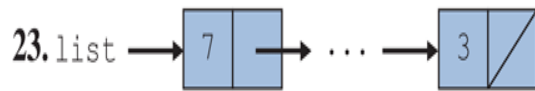
Section 16.2: A Linked List Class

18. What are the two ways to change the contents of a linked list?
19. An element can be inserted at or removed from the beginning, middle, or end of a linked list. Which of the three locations is the most computationally expensive, and why? How does this compare against the result for an array list?
20. When you add or remove the element found at index i of a list, you must create a temporary `current` node reference and advance it through the list. At which index's node should the loop stop, relative to i ?
21. In an array list, it is possible to overrun the capacity of the array, at which point the list must be resized to fit. Is resizing necessary on a linked list? What limits the number of elements that a linked list can have?

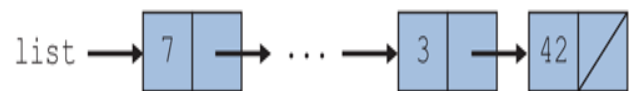
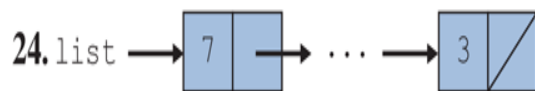
For each of the next three problems, you'll see pictures of long chains of linked nodes before and after changes. (The . . . in the middle of the chain signifies an indeterminate large number of nodes.) Write the code that will produce the given result by modifying links between the nodes shown and/or creating new nodes as needed. You will need to write loops to advance to the end of the list in order to reach the node(s) to modify.



(change all nodes to store the value 42)



(change the last node to store the value 42)



(add a node at the end that stores the value 42)

25. Write methods called `min` and `max` that return the smallest and largest values in the linked list, respectively. For example, if a variable called `list` stores `[11, -7, 3, 42, 0, 14]`, the call of `list.min()` should return `-7` and the call of `list.max()` should return `42`. If the list is empty, throw an `IllegalStateException`.

Section 16.3: A Complex List Operation

26. What are the four cases examined in the `addSorted` method?
27. What is the “inchworm approach”? What advantages does this approach have over other approaches for examining a linked list?
28. Write methods called `sum` and `average` that return the sum of all values in the list and the average value as a real number, respectively. For example, if a variable called `list` stores `[11, -7, 3, 42, 0, 14]`, the call of `list.sum()` should return `63` and the call of `list.average()` should return `10.5`. If the list is empty, `sum` should return `0` and `average` should return `0.0`.

Section 16.4: An `IntList` Interface

29. What are some advantages of creating an `IntList` interface and having both types of lists implement it?
30. Write a method called `firstLast` that can accept either type of integer list as a parameter and that moves the first element of the list to the end. For example, if a variable called `list` contains the values `[12, 45, 78, 20, 36]`, the call of `firstLast(list);` will change the list to store `[45, 78, 20, 36, 12]`.

Section 16.5: `LinkedList<E>`

31. What are some changes that need to be made to the linked list class to convert it from storing integers to storing objects of type `E`?
32. Why is an iterator especially useful with linked lists?
33. What state does the linked list iterator store? How does the iterator know if there are more elements left to examine?

Exercises

Each of the following exercises is a method to be added to the `LinkedList` class from this chapter.

1. Write a method called `set` that accepts an index and a value and sets the list's element at that index to have the given value. You may assume that the index is between 0 (inclusive) and the size of the list (exclusive).
2. Write a method called `min` that returns the minimum value in a list of integers. If the list is empty, it should throw a `NoSuchElementException`.
3. Write a method called `isSorted` that returns `true` if the list is in sorted (nondecreasing) order and returns `false` otherwise. An empty list is considered to be sorted.
4. Write a method called `lastIndexOf` that accepts an integer value as a parameter and that returns the index in the list of the last occurrence of that value, or `-1` if the value is not found in the list. For example, if a variable `list` stores the values `[1, 18, 2, 7, 18, 39, 18, 40]`, then the call of `list.lastIndexOf(18)` should return `6`. If the call had instead been `list.lastIndexOf(3)`, the method would return `-1`.
5. Write a method called `countDuplicates` that returns the number of duplicates in a sorted list. The list will be in sorted

order, so all of the duplicates will be grouped together. For example, if a variable `list` stores the values `[1, 1, 1, 3, 3, 6, 9, 15, 15, 23, 23, 23, 40, 40]`, the call of `list.countDuplicates()` should return `7` because there are 2 duplicates of `1`, 1 duplicate of `3`, 1 duplicate of `15`, 2 duplicates of `23`, and 1 duplicate of `40`.

6. Write a method called `hasTwoConsecutive` that returns whether or not a list of integers has two adjacent numbers that are consecutive integers (`true` if such a pair exists and `false` otherwise). For example, if a variable `list` stores the values `[1, 18, 2, 7, 8, 39, 18, 40]`, then the call `list.hasTwoConsecutive()` should return `true` because the list contains the adjacent numbers `(7, 8)`, which are a pair of consecutive numbers.
7. Write a method called `deleteBack` that deletes the last value (the value at the back of the list) and returns the deleted value. If the list is empty, throw a `NoSuchElementException`.
8. Write a method called `switchPairs` that switches the order of values in the list in a pairwise fashion. Your method should switch the order of the first two values, then switch the order of the next two, switch the order of the next two, and so on. If the list contains an odd number of values, the final element is not moved. For example, if the list initially stores `[10, 25, 31, 47, 52, 68, 77]`, your method should switch the first pair (`10` and `25`), the second pair (`31` and `47`), and the third pair (`52` and `68`) to yield `[25, 10, 47, 31, 68, 52, 77]`.

9. Write a method called `stutter` that doubles the size of a list by replacing every integer in the list with two of that integer. For example, suppose a variable `list` stores the values `[1, 8, 19, 4, 17]`, after a call of `list.stutter()`, it should store `[1, 1, 8, 8, 19, 19, 4, 4, 17, 17]`.
10. Write a method called `stretch` that takes an integer n as a parameter and that increases a list of integers by a factor of n by replacing each integer in the original list with n copies of that integer. For example, if a variable called `list` stores `[18, 7, 4, 24, 11]` and we make the call of `list.stretch(3)`; the list should be changed to store `[18, 18, 18, 7, 7, 7, 4, 4, 4, 24, 24, 24, 11, 11, 11]`. If n is zero or negative, the list should become empty.
11. Write a method called `compress` that replaces every pair of elements in the list with a single element equal to the sum of the pair. If the list is of odd size, leave the last element unchanged. For example, if the list stores `[1, 7, 3, 9, 4, 6, 5]`, your method should change it to store `[8, 12, 10, 5]` ($1 + 7$, then $3 + 9$, then $4 + 6$, then 5).
12. Write a method called `split` that rearranges the elements of a list so that all of the negative values appear before all of the nonnegatives. For example, suppose a variable `list` stores the values `[8, 7, -4, 19, 0, 43, -8, -7, 2]`. The call of `list.split()` should rearrange the list to put the negatives first: `[-4, -8, -7, 8, 7, 19, 0, 43, 2]`. It doesn't matter what order the numbers are in, only that the negatives appear

before the nonnegatives, so this is only one possible solution. You must solve the problem by rearranging the links of the list, not by swapping `data` values or creating new nodes. You also may not use auxiliary structures like arrays or `ArrayLists` to solve this problem.

13. Write a method called `transferFrom` that accepts a second linked list as a parameter and that moves values from the second list to this list. You are to attach the second list's elements to the end of this list. You are also to empty the second list. For example, suppose two lists called `list1` and `list2` store `[8, 17, 2, 4]` and `[1, 2, 3]`, respectively. The call of `list1.transferFrom(list2);` should change `list1` to `[8, 17, 2, 4, 1, 2, 3]` and `list2` to an empty list, `[]`. The order of the arguments matters; `list2.transferFrom(list1);` should change `list1` to an empty list, `[]`, and `list2` to `[1, 2, 3, 8, 17, 2, 4]`. Either of the two lists could be empty, but you can assume that neither list is `null`. You are not to create any new nodes. Your method should simply change links of the lists to join them together.
14. Write a method called `removeAll` that removes all occurrences of a particular value. For example, if a variable `list` stores the values `[3, 9, 4, 2, 3, 8, 17, 4, 3, 18]`, the call of `list.removeAll(3);` would change the list to store `[9, 4, 2, 8, 17, 4, 18]`.
15. Write a method called `equals` that accepts a second list as a parameter, returns `true` if the two lists are equal, and returns

`false` otherwise. Two lists are considered equal if they store exactly the same values in exactly the same order and have exactly the same length.

16. Write a method called `removeEvens` that removes the values in even-numbered indexes from a list, returning a new list that contains those values in their original order. For example, consider a variable `list1` that stores the values `[8, 13, 17, 4, 9, 12, 98, 41, 7, 23, 0, 92]` and imagine that the following call is made:

```
LinkedList list2 = list1.removeEvens();
```

After the call, `list1` should store `[13, 4, 12, 41, 23, 92]` and `list2` should store `[8, 17, 9, 98, 7, 0]`. You may not call any methods of the class other than the constructor to solve this problem. You may not create any new nodes nor change the values stored in `data` fields to solve this problem. You must solve it by rearranging the links of the list.

17. Write a method called `removeRange` that accepts a starting and ending index as parameters and removes the elements at those indexes (inclusive) from the list. For example, if a variable `list` stores the values `[8, 13, 17, 4, 9, 12, 98, 41, 7, 23, 0, 92]`, the call of `list.removeRange(3, 8);` should remove values between index 3 and index 8 (the values `4` and `7`), leaving the list of `[8, 13, 17, 23, 0, 92]`. The method

should throw an `IllegalArgumentException` if either of the positions is negative. Otherwise you may assume that the positions represent a legal range of the list ($0 \leq \text{start} \leq \text{end} < \text{size}$).

18. Write a method called `doubleList` that doubles the size of a list by appending a copy of the original sequence to the end of the list. For example, if a variable `list` stores the values `[1, 3, 2, 7]` and we make the call of `list.doubleList();` then after the call it should store `[1, 3, 2, 7, 1, 3, 2, 7]`. Notice that the list has been doubled in size by having the original sequence appear twice in a row. You may not make assumptions about how many elements are in the list. You may not call any methods of the class to solve this problem. If the original list contains N nodes, then you should construct exactly N nodes to be added. You may not use any auxiliary data structures such as arrays or `ArrayLists` to solve this problem. Your method should run in $O(N)$ time where N is the number of nodes in the list.
19. Write a method called `rotate` that moves the value at the front of a list of integers to the end of the list. For example, if a variable called `list` stores the values `[8, 23, 19, 7, 45, 98, 102, 4]`, then the call of `list.rotate();` should move the value `8` from the front of the list to the back of the list, changing the list to store `[23, 19, 7, 45, 98, 102, 4, 8]`. If the method is called for a list of 0 elements or 1 element, it should have no effect on the list. You may neither construct any new nodes to solve this problem nor change any of the

`data` values stored in the nodes. You must solve the problem by rearranging the links of the list.

20. Write a method called `shift` that rearranges the elements of a list of integers by moving to the end of the list all values that are in odd-numbered positions and otherwise preserving list order. For example, suppose that a variable `list` stores the values `[10, 31, 42, 23, 44, 75, 86]`. The call of `list.shift();` should rearrange the list to store `[10, 42, 44, 86, 31, 23, 75]`. It doesn't matter whether the value itself is odd or even; what matters is whether the value appears in an odd index (index 1, 3, 5, etc.). Also notice that the original order of the list is otherwise preserved. You may not construct any new nodes nor use any auxiliary data structures to solve this problem. You also may not change any `data` fields of the nodes; you must solve this problem by rearranging the links of the list.
21. Write a method called `surroundWith` that takes an integer `x` and an integer `y` as parameters and surrounds all nodes in the list containing the value `x` with new nodes containing the value `y`. In particular, each node that contains the value `x` as data should have a new node just before it and just after it that each contain the value `y`. If no nodes in the list contain the value `x`, then the list should not be modified. For example, suppose that the variables `list1`, `list2`, and `list3` store the following sequences of values:

```
[0, 1, 2, 1]    // stored in list1
[0, 1, 0]       // stored in list2
[0, 1, 2]       // stored in list3
```

and we make the following calls:

```
list1.surroundWith(1, 4);    // surround 1s with 4s
list2.surroundWith(1, 1);    // surround 1s with 1s
list3.surroundWith(3, 4);    // surround 3s with 4s
```

then the variables will now store these sequences:

```
[0, 4, 1, 4, 2, 4, 1, 4]    // stored in list1
[0, 1, 1, 1, 0]             // stored in list2
[0, 1, 2]                   // stored in list3
```

22. Write a method called `reverse` that reverses the order of the elements in the list. (This is very challenging!) For example, if the variable `list` initially stores the values `[1, 8, 19, 4, 17]`, the call of `list.reverse();` should change the list to store `[17, 4, 19, 8, 1]`.

Programming Projects

1. The actual `List` interface in the `java.util` package has several methods beyond the ones implemented in this chapter. Write a version of `LinkedList<E>` that adds some or all of these methods. The methods to add are the following (some headers are slightly modified; see the Java API Specification for descriptions of each method):

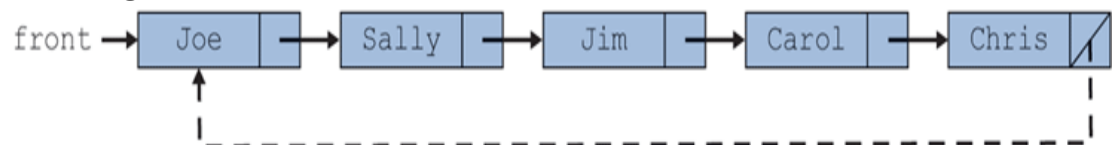
- `public void addAll(int index, List<E> list)`
- `public boolean containsAll(List<E> list)`
- `public boolean equals(Object o)`
- `public int lastIndexOf(Object o)`
- `public boolean remove(Object o)`
- `public void removeAll(List<E> list)`
- `public void retainAll(List<E> list)`
- `public Object[] toArray()`

2. The `java.util` package has an interface called `ListIterator` that extends the `Iterator` interface and includes additional methods that are specific to iterating through the elements of lists forward or backward. Write a class called `LinkedListIterator2` that adds some or all of these methods for iterating over a doubly linked list. The methods to add are

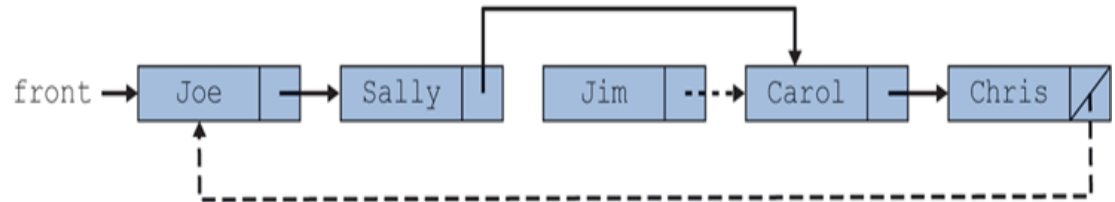
the following (see the Java API Specification for descriptions of each method):

- `public void add(E value)`
- `public boolean hasPrevious()`
- `public int nextIndex()`
- `public E previous()`
- `public int previousIndex()`
- `public void set(E value)`

3. The implementation of several methods is (or can be) the same between our `ArrayList` and `LinkedList`. Write a common abstract superclass called `AbstractList` that implements the common behavior and is extended by both `ArrayList` and `LinkedList`. Factor out the common code from the two list classes into this abstract superclass so that no code duplication occurs between the two. Use iterators wherever possible in the abstract code to ensure that the implementation is efficient for both types of lists.
4. “Assassin” is a real-life game in which a group of players all try individually to find and touch (or “kill”) one other player. You can use a linked list to represent this “kill ring” of players in the game:

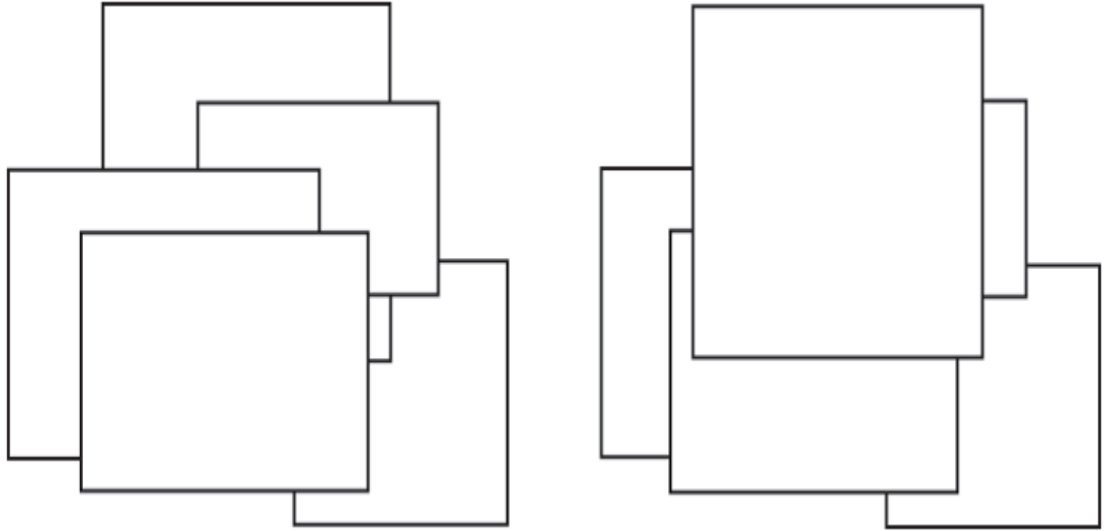


If a “kill” is made, the ring adjusts by removing that person from the list. For example, the following occurs if Sally kills Jim:



Write a program that models a game of Assassin. The game reads the names of the initial kill ring from a file and puts them into a linked list in random order. Then the game repeatedly prompts for the name of a person that has been killed. The game continues until only one player remains and is declared the winner. The program should also have methods for printing the current contents of the kill ring and printing a “graveyard” of all players who have been killed.

5. Write a graphical program that shows a set of overlapping rectangular regions, each in a different random color. The regions are represented internally as a linked list. The regions have a “z-ordering” in which rectangles closer to the end of the list are closer to the top, closer to the user’s field of view. When the user clicks the mouse, the topmost region that touches the mouse pointer moves to the very top (end) of the linked list. For example, the following diagrams show the top-left rectangle before and after the mouse is clicked:



(See the Nifty Assignments page at <http://nifty.stanford.edu> for a more detailed description of this project, as written by its original author, Prof. Michael Clancy of the University of California, Berkeley.)